

+ 26

Back To Basics

Function Overloading

CHRIS RYAN



20
26



Back to Basics: Function (and Operator) Overloading

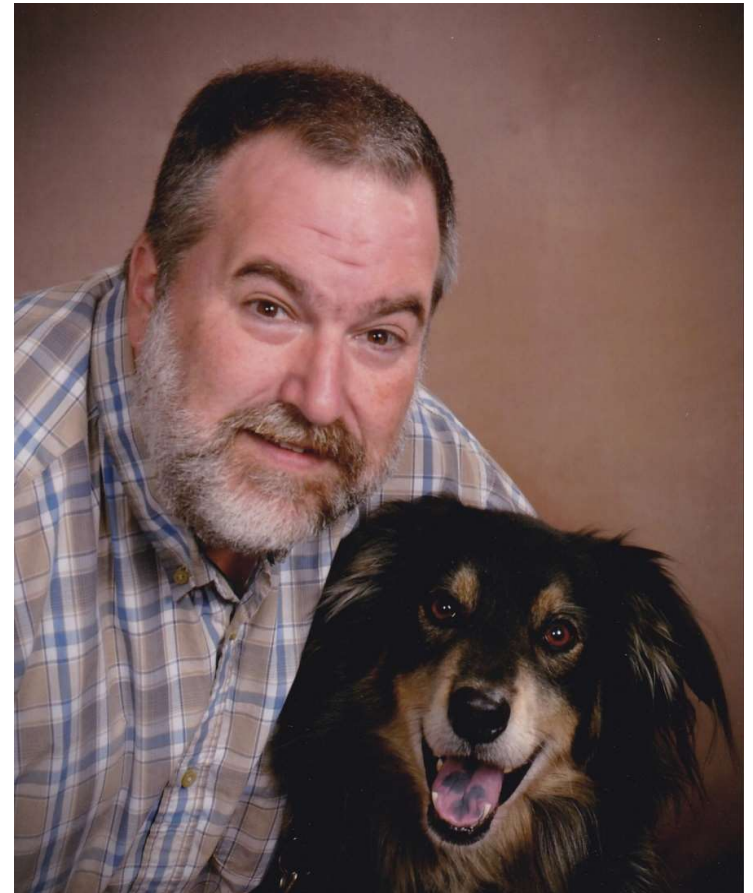


B2B: Function Overloading – about me

About Me: Chris Ryan, Palm Springs area, Ca, USA
[Linkedin.com/in/chrisr98008](https://www.linkedin.com/in/chrisr98008)
(No blind invites! Tell me how we know each other)

- Classically trained in Computer Systems Engineering (SW & HW)
- Well experienced in Classic 'C', Classic C++, & Modern C++
- WG21 / ISO C++ Standards Committee member, EWG
- Large Scale, **Complex** Problem Spaces, **Simple** Solutions
- Firmware & Embedded (large & small)
- Prefer Windows, because that is what I know
- Win32/64 SDK/MFC, Firmware/Embedded, Bare Metal, Linux
- Worked in many domains, for many companies:
 - Number Crunching
 - Business / Internet
 - Networking / Protocols
 - Low-Level Operating System Internals
 - Data Engines
 - Scientific / Medical
 - Engineering / Robotics

I believe in reducing complexity through simplification



B2B: Function Overloading –

Warning contains: (Yes, I know what that stands for)

Foo / Bar
& Slideware

(California Proposition 65)

B2B: Function Overloading – what is overloading

Compile-Time Polymorphism

The C++ compiler selects the correct function to invoke by matching the arguments provided in the function call to the function signatures. Do the right thing, for what I have.

1. Different Number of Parameters
2. Different Types of Parameters
3. Different Sequence of Parameters
4. Rules and Common Pitfalls
 1. Return types do not count
 2. Ambiguity with default arguments

B2B: Function Overloading – number of parameters

Different Number of Parameters

Varying the total count of arguments they accept.

```
int add(int a, int b) { return a + b;}
```

```
int add(int a, int b, int c) { return a + b + c;}
```

```
int main()
{
    std::cout << add(5, 10) << std::endl; // Calls the 2-parameter version (Outputs 15)
    std::cout << add(5, 10, 15) << std::endl; // Calls the 3-parameter version (Outputs 30)
}
```

B2B: Function Overloading – types of parameters

Different Types of Parameters

Same number of parameters but change their data types.

```
void printValue(int i)          { std::cout << "Integer: " << i << '\n'; }
void printValue(double d)      { std::cout << "Double: " << d << '\n'; }
void printValue(std::string s) { std::cout << "String:  " << s << '\n'; }
```

```
int main()
{
    printValue(42);           // Calls printValue(int)
    printValue(3.14);        // Calls printValue(double)
    printValue("Hello C++"); // Calls printValue(string)
}
```

B2B: Function Overloading – sequence of parameters

Different Sequence of Parameters

Multiple arguments of different types, different order

```
void displayData(int id, double salary)
{
    std::cout << "ID: " << id << ", Salary: " << salary << std::endl;
}
```

```
void displayData(double salary, int id)
{
    std::cout << "Salary: " << salary << ", ID: " << id << std::endl;
}
```

```
int main()
{
    displayData(101, 5500.50); Calls the first version
    displayData(5500.50, 101); Calls the second version
}
```

B2B: Function Overloading – rules and pitfalls

Rules and Common Pitfalls

Must follow these rules to avoid compiler errors

- Return types do not count

```
int    getValue() { return 10; }  
Double getValue() { return 5.5; } ❌ // Compiler Error: Duplicate declaration
```

- Ambiguity with Default Arguments

```
void compute(int x)          { }  
void compute(int x, int y = 0) { }
```

B2B: Function Overloading – why use overloading

In C++, use overloading when you want to use **conceptually identical operations across a different set of data.**

What is cleaner and more obvious:

```
printInt(int i)
printString(string str)
```

Or:

```
print(int i)
print(string str)
```

Later, you can add:

```
print(T& t [, ...])
```

This makes code highly readable, natural and intuitive.

B2B: Function Overloading – twins but not



B2B: Function Overloading – not twins

Free Functions

```
void Foo()  
void Foo(int i)  
void Foo(double d)  
void Foo(std::string s)
```

```
int main()  
{
```

```
    Foo();  
    Foo(1);  
    Foo(2.3);  
    Foo("hello");  
}
```

```
{ std::print("Foo()      \n"); }  
{ std::print("Foo({})   \n", i); }  
{ std::print("Foo({})   \n", d); }  
{ std::print("Foo(\"{} \") \n", s); }
```

Output:

```
Foo()  
Foo(1)  
Foo(2.3)  
Foo("hello")
```

godbolt.org/z/bnjTGv1vb

B2B: Function Overloading – twins not

Member Functions

```
struct Bar
{
    // data members, ...
```

```
    void Foo()
    void Foo(int i)
    void Foo(double d)
    void Foo(std::string s)
```

```
};
int main()
{
    Bar bar;
    bar.Foo();
    bar.Foo(1);
    bar.Foo(2.3);
    bar.Foo("hello");
}
```

```
{ std::print("Bar::Foo()          \n"); }
{ std::print("Bar::Foo({})       \n", i); }
{ std::print("Bar::Foo({})       \n", d); }
{ std::print("Bar::Foo(\"{} \") \n", s); }
```

Output:

```
Bar:: Foo()
Bar:: Foo(1)
Bar:: Foo(2.3)
Bar:: Foo("hello")
```

godbolt.org/z/Ps86vWboY

B2B: Function Overloading – name mangling



B2B: Function Overloading – oversimplified name mangling

Free Functions

```
void Foo()
void Foo(int i)
void Foo(double d)
void Foo(std::string s)

int main()
{
    Foo();
    Foo(1);
    Foo(2.3);
    Foo("hello");
}
```

```
{ std::print("Foo()      \n"); }
{ std::print("Foo({})    \n", i); }
{ std::print("Foo({})    \n", d); }
{ std::print("Foo(\"{} \") \n", s); }
```

Output:

```
Foo()
Foo(1)
Foo(2.3)
Foo("hello")
```

godbolt.org/z/bnjTGv1vb

B2B: Function Overloading – oversimplified name mangling

Free Functions

```
void Foo_v ()  
void Foo_i (int i)  
void Foo_d (double d)  
void Foo_str(std::string s)
```

```
int main()  
{
```

```
    Foo_v ();  
    Foo_i (1);  
    Foo_d (2.3);  
    Foo_str(std::string("hello"));  
}
```

```
{ std::print("Foo()      \n"); }  
{ std::print("Foo({})   \n", i); }  
{ std::print("Foo({})   \n", d); }  
{ std::print("Foo(\"{} \") \n", s); }
```

Output:

```
Foo()  
Foo(1)  
Foo(2.3)  
Foo("hello")
```

B2B: Function Overloading – oversimplified name mangling

Member Functions

```
struct Bar
{
    // data members, ...
```

```
    void Foo()
    void Foo(int i)
    void Foo(double d)
    void Foo(std::string s)
```

```
};
int main()
{
    Bar bar;
    bar.Foo();
    bar.Foo(1);
    bar.Foo(2.3);
    bar.Foo("hello");
}
```

```
{ std::print("Bar::Foo()      \n"); }
{ std::print("Bar::Foo({})    \n", i); }
{ std::print("Bar::Foo({})    \n", d); }
{ std::print("Bar::Foo(\"{} \") \n", s); }
```

Output:

```
Bar:: Foo()
Bar:: Foo(1)
Bar:: Foo(2.3)
Bar:: Foo("hello")
```

godbolt.org/z/Ps86vWboY

B2B: Function Overloading – oversimplified name mangling

Member Functions

```
struct Bar
{
    // data members, ...
};

void Bar_Foo_v (Bar* this)           { std::print("Bar::Foo()      \n"); }
void Bar_Foo_i (Bar* this, int i)    { std::print("Bar::Foo({})   \n", i); }
void Bar_Foo_d (Bar* this, double d) { std::print("Bar::Foo({})   \n", d); }
void Bar_Foo_str(Bar* this, std::string s) { std::print("Bar::Foo(\"{} \") \n", s); }

int main()
{
    Bar bar;
    Bar_Foo_v (&bar);
    Bar_Foo_i (&bar, 1);
    Bar_Foo_d (&bar, 2.3);
    Bar_Foo_str(&bar, std::string("hello"));
}
```

Output:

```
Bar::Foo()
Bar::Foo(1)
Bar::Foo(2.3)
Bar::Foo("hello")
```

B2B: Function Overloading – rules

- **Number of parameters:** One function takes two arguments, another takes three.
- **Type of parameters and count:** One function takes an int, while another takes a double or string.
- **Sequence (order) of parameters:** For example,
 - func(int, double) is different from
 - func(double, int).
- You **cannot** overload a function by changing only its **return type**

B2B: Function Overloading – your first exposure

```
struct Point
{
    int x{}, y{};

    Point(){}
    Point(int x, int y) : x(x), y(y) {}
};

struct Rect
{
    int left{}, top{}, right{} , bot{};

    Rect(){}
    Rect(int left, int top, int right , int bot)
        : left(left), top(top), right(right) , bot(bot) {};

    Rect(const &Point topleft, const & Point botright)
        : left(topleft.x), top(topleft.y), right(botright.x) , bot(botright.y) {};
};
```

B2B: Function Overloading – deeper example

```
struct Point
{
    int x{}, y{};

    Point() {}
    Point(const Point &rhs) : x(rhs.x), y(rhs.y) {};
    Point(int x, int y) : x(x), y(y) {};
};

struct Rect
{
    Point topleft{}, bottomRight{};

    Rect() {}
    Rect(const Rect & rhs) : topleft(rhs.topleft), bottomRight(rhs.bottomRight) {}
    Rect(const Point &topleft, const Point &bottomRight) : topleft(topleft), bottomRight(bottomRight) {}

    Rect(int t, int l, int bottom, int right) : topleft(Point(l,t)), bottomRight(Point(right,bottom)) {}
    Rect(int t, int l, int width, int height) : topleft(Point(t,l)), bottomRight(Point(l+width,t+height)) {}
    Rect(const Point &topleft, int w, int h) : topleft(topleft), bottomRight(Point(topleft.x+w, topleft.y+h)) {}
};

Rect myRect(Point(left, top), height, width);
```

godbolt.org/z/sf7E3fzn

B2B: Function Overloading – example part 1/2

```
struct Point
{
    int x{}, y{};

    Point() {}
    Point(int x, int y) : x(x), y(y) {};

    Point(const Point &rhs) : x(rhs.x), y(rhs.y) {};
};

int main()
{
    Point p0;
    Point p1{10, 20};
    Point p2(p1);
    Point p3{30, 40};
    //...
}
```

godbolt.org/z/sf7E3fznb

B2B: Function Overloading – example part 2/2

```
struct Rect
{
    Point topleft{}, bottomRight{};

    Rect() {}
    Rect(const Point &topleft, const Point &bottomRight) : topleft(topleft), bottomRight(bottomRight) {}
    Rect(int top, int left, int right, int bottom) : topleft(Point(left,top)), bottomRight(Point(right,bottom)) {}
Rect(int top, int left, int width, int height) : topleft(Point(top,left)), bottomRight(Point(left+width,top+height)) {}
    Rect(const Point &topleft, int width, int height) : topleft(topleft), bottomRight(Point(topleft.x+width, topleft.y+height)) {}
    Rect(const Rect & rhs) : topleft(rhs.topleft), bottomRight(rhs.bottomRight) {}
};

int main()
{
    Point p1{10, 20};
    Point p3{30, 40};
    //...
    Rect rect0;
    Rect rect1(p1, p3);
    Rect rect2(rect1);
    Rect rect3(10, 20, 30, 40); //top, left, right, bottom
Rect rect4(10, 20, 20, 20); //top, left, width, height
    Rect rect5(p1, 20, 20);
}
```

godbolt.org/z/sf7E3fznb

B2B: Function Overloading – least surprise

"Doctrine of Least Surprise": Standard C++

- **Don't Change Meanings:** Avoid making + perform subtraction; it confuses other developers and makes code unpredictable.
- **Performance Clarity:** An overloaded operator might hide an expensive operation (like deep copying a large matrix or opening a new database connection), making it harder to spot performance bottlenecks at a glance.
- **Limited Customization:** you cannot create *new* symbols or change the precedence and associativity of existing operators.

B2B: Function Overloading – why overloading

Why Function Overloading is Useful

- **Improves Code Readability and Cleanliness**

Without Overloading:	<code>printInt(int x)</code> <code>printDouble(double x)</code> <code>printString(string x)</code>
With Overloading:	<code>print(x)</code>

- **Enables Generic-Feeling Behavior for Different Types**

```
int add(int a, int b)    {return a + b; }  
double add(double a, double b) {return a + b; }  
  
int main()  
{  
    std::cout << add(5, 10);           // Calls int overload  
    std::cout << add(5.5, 2.3);        // Calls double overload  
}
```

B2B: Function Overloading – why overloading (cont)

- **Permits Flexible Function Signatures (Varying Argument Counts & Types)**

Draw a point with just coordinates

```
void drawPoint(int x, int y);
```

Draw a point with coordinates and a custom color

```
void drawPoint(int x, int y, std::string color);
```

- **Crucial for Constructor Overloading**

See the Point/Rect previous examples

B2B: Function Overloading – opportunity for overloading

```
int main()
{
    std::list<int> myList {10, 20, 30};
    for(auto it= myList.begin(); it!= myList.end(); it++)
        std::cout << *it << " ";
    std::cout << "\n";

    std::vector<int> myVector {100, 200, 300};
    for(auto it= myVector.begin(); it!= myVector.end(); it++)
        std::cout << *it << " ";
    std::cout << "\n";

    int myArray[] {1000,2000,3000};
    for(const auto& number : myArray)
        std::cout << number << " ";
    std::cout << "\n";
}
```

Output:

10 20 30

100 200 300

1000 2000 3000

godbolt.org/z/MdTaGPGWf

B2B: Function Overloading – overloading using a template

```
template<typename T>
void process(const T& t)
{
    for(auto it=t.begin(); it!=t.end(); it++)
        std::cout << *it << " ";
    std::cout << "\n";
}

int main()
{
    std::list<int>    myList    {10,  20,  30};
    std::vector<int> myVector  {100, 200, 300};
    int              myArray[] {1000,2000,3000};

    process(myList);
    process(myVector);
    process(myArray);
}
```

Output:

```
10 20 30
100 200 300
1000 2000 3000
```

godbolt.org/z/nxW1d43aP

B2B: Function Overloading – overloading using concepts

```
template <typename T>
concept Iterable = requires(T t) { t.begin(); t.end(); };

void process(const Iterable auto& iterable) {
    for(auto it=iterable.begin(); it!=iterable.end(); it++)
        std::cout << *it << " ";
    std::cout << "\n";
}

int main()
{
    std::list<int>    myList    { 10, 20, 30};
    std::vector<int> myVector   { 100, 200, 300};
    int              myArray[] {1000,2000,3000};

    process(myList);
    process(myVector);
    // process(myArray); //error not iterable?

    for(const auto& number : myArray)
        std::cout << number << " ";
    std::cout << "\n";
}
```

Output:

10 20 30

100 200 300

1000 2000 3000

godbolt.org/z/h3WhxsWr5

B2B: Function Overloading – overloading using s

```
template <typename T>
concept Iterable = requires(T t) { t.begin(); t.end(); };

void process(const Iterable auto& iterable) {
    for(auto it=iterable.begin(); it!=iterable.end(); it++)
        std::cout << *it << " ";
    std::cout << "\n";
}

template <typename T>
void process(const T& non_iterable) requires (!Iterable<T>)
{
    for(const auto& item : non_iterable)
        std::cout << item << " ";
    std::cout << "\n";
}

int main()
{
    std::list<int>    myList    { 10, 20, 30};
    std::vector<int> myVector { 100, 200, 300};
    int              myArray[] {1000,2000,3000};
    process(myList);
    process(myVector);
    process(myArray);
}
```

Output:

10 20 30

100 200 300

1000 2000 3000

godbolt.org/z/sc9E6orrK

B2B: Function Overloading – dumb example

```
struct QueryFilters { std::string role; bool isActive; };

class Database
{
    void connect(const Database &database)          { std::cout << "Copy DB connection\n"; }
    void connect(std::string server, std::string table)
        { std::cout << std::format("Connect to server '{}', table '{}'\n", server, table); }
    void select(std::string query)                  { std::cout << query << '\n'; }
public:
    Database()                                     { std::cout << "Empty DB\n"; }
    Database(const Database &database)              { connect(database); }
    Database(std::string server, std::string table) { connect(server, table); }

    void fetchUser(int userId)
        { select(std::format("SQL: SELECT * FROM users WHERE ID = {}", userId)); }
    void fetchUser(const std::string& username)
        { select(std::format("SQL: SELECT * FROM users WHERE UN = {}", username)); }
    void fetchUser(const QueryFilters& qf)
        { select(std::format("SQL: SELECT * FROM users WHERE role = {}/{})", qf.role, qf.isActive)); }
};

int main()
{
    Database db("MyDB", "Mytable");
    db.fetchUser(1042);
    db.fetchUser("john_doe");
    db.fetchUser(QueryFilters{"Admin", true});
}
```

Output:

Connect to server 'MyDB' table 'Mytable'

SQL: SELECT * FROM users WHERE ID = 1042

SQL: SELECT * FROM users WHERE UN = john_doe

SQL: SELECT * FROM users WHERE role = Admin/true

godbolt.org/z/Wxsh7Gh4Y

B2B: Function Overloading – dumb example 2

```
struct Point { int x{}, y{}; };

struct Renderer
{
void draw(Point center, int radius)
    { std::cout << std::format("Rendering circle at:({},{}) with radius:{}\n", center.x, center.y, radius); }

    void draw(Point tl, int width, int height)
        { std::cout << std::format("Rendering rectangle at:({},{}) width:{} height:{}\n", tl.x, tl.y, width, height);}

    void draw(const char* text, Point position)
        { std::cout << std::format("Rendering text \"{}\" at position:({},{})\n", text, position.x, position.y); }
};

int main() {
    Renderer engine;
    Point p{10, 20};

    engine.draw(p, 5);
    engine.draw(p, 100, 50);
    engine.draw("Game Over", p);
}
```

Output:

```
Rendering circle at:(10,20) with radius:5
Rendering rectangle at:(10,20) width:100 height:50
Rendering text "Game Over" at position:(10,20)
```

godbolt.org/z/Wxsh7Gh4Y

B2B: Function Overloading – failed example 3

```
struct CreditCard { std::string cardNumber, expirationDate; int cvv; };

struct PaymentGateway
{
    void processPayment(double amount, const CreditCard& card)
    { std::cout << "Charging $" << amount << " to Credit Card ending in "
      << card.cardNumber.substr(card.cardNumber.length() - 4) << "\n"; }

    void processPayment(double amount, std::string email)
    { std::cout << "Requesting $" << amount << " via PayPal account: " << email << "\n"; }

    void processPayment(double amount, const char* cryptoWalletAddress)
    { std::cout << "Awaiting $" << amount << " to blockchain address: " << cryptoWalletAddress << "\n"; }
};

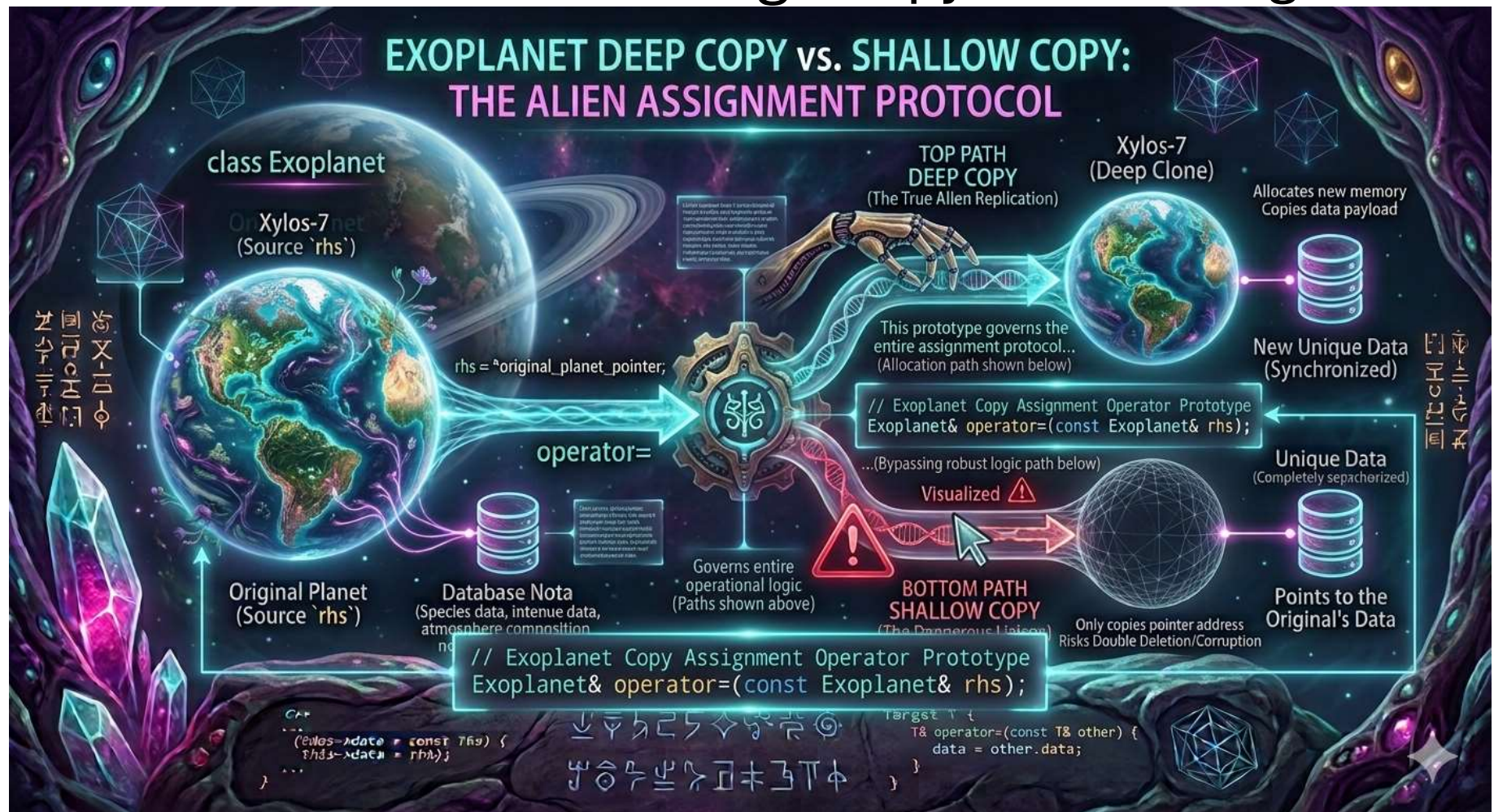
int main() {
    PaymentGateway gateway;
    CreditCard myCard{"1234567890123456", "12/29", 999};

    gateway.processPayment(49.99, myCard);
    gateway.processPayment(15.50, "user@example.com");
    gateway.processPayment(1200.00, "0x71C...3a9");
}
```

Output:
Charging \$49.99 to Credit Card ending in 3456
Awaiting \$15.5 to blockchain address: user@example.com
Awaiting \$1200 to blockchain address: 0x71C...3a9

godbolt.org/z/Wxsh7Gh4Y

B2B: Function Overloading – copy c'tor & assignment



B2B: Function Overloading – deep subject

```
struct Exoplanet
{
    using VStrings = std::vector<std::string>;
    std::string name;
    VStrings oceans, land;

    Exoplanet(){}
    Exoplanet(std::string name, VStrings oceans, VStrings land) : name(name), oceans(oceans), land(land){}

    Exoplanet(const Exoplanet& other) { DeepCopy (other); }
    Exoplanet& operator=(const Exoplanet& other) { return ShallowCopy(other); }

    Exoplanet& DeepCopy(const Exoplanet& other)
    { name = other.name; oceans = other.oceans; land = other.land; return *this; }

    Exoplanet& ShallowCopy(const Exoplanet& other)
    { name = other.name; oceans.clear(); land.clear(); return *this; }

    friend std::ostream& operator<<(std::ostream& os, const Exoplanet& rhs)
    {
        auto List = [](const auto& vect){ for(auto const v : vect) std::cout << v << ", "; };
        std::cout << "Name: " << rhs.name << ": \n";
        std::cout << "Oceans: "; List(rhs.oceans); std::cout << '\n';
        std::cout << "Land: "; List(rhs.land); std::cout << '\n';
        return os; }
};
```

godbolt.org/z/7zbrd17vh

B2B: Function Overloading – you're so shallow

```
struct Exoplanet
{
    Exoplanet(std::string name, VStrings oceans, VStrings land) : name(name), oceans(oceans), land(land){}
    Exoplanet(const Exoplanet& other) { DeepCopy (other); }
    Exoplanet& operator=(const Exoplanet& other) { return ShallowCopy(other); }

    Exoplanet& DeepCopy (const Exoplanet& other)
        { name = other.name; oceans = other.oceans; land = other.land; return *this; }
    Exoplanet& ShallowCopy(const Exoplanet& other)
        { name = other.name; oceans.clear(); land.clear(); return *this; }

    friend std::ostream& operator<<(std::ostream& os, const Exoplanet& other)
    {
        os << "Exoplanet: Name: " << other.name << "\n";
        os << "Oceans: " << other.oceans << "\n";
        os << "Land: " << other.land << "\n";
    }
};

int main()
{
    Exoplanet shallowcopy{};
    Exoplanet celestia("Celestia", {"Abyss", "Azure"}, {"Spire", "Crown", "Meadows"});

    Exoplanet deepcopy = celestia;
    shallowcopy = celestia;

    std::cout << "Exoplanet: " << celestia << '\n';
    std::cout << "deep copy: " << deepcopy << '\n';
    std::cout << "shallow copy: " << shallowcopy << '\n';
}
```

Output:

Exoplanet: Name: Celestia:
Oceans: Abyss, Azure,
Land: Spire, Crown, Meadows,

deep copy: Name: Celestia:
Oceans: Abyss, Azure,
Land: Spire, Crown, Meadows,

shallow copy: Name: Celestia:
Oceans:
Land:

godbolt.org/z/7zbrd17vh

B2B: Function Overloading – typical base/derived classes

```
struct Person
{
    std::string name{"empty"};
    int age{};

    Person(std::string name, int age) : name(name), age(age) {}
    Person(const Person& other)       : Person(other.name, other.age) {}
    Person& operator=(const Person& other)
    { name = other.name; age = other.age; return *this; }

    friend std::ostream& operator<<(std::ostream& os, const Person& rhs) {...}
};

struct Student : public Person
{
    float gpa{};

    Student(std::string name, int age, float gpa) : Person(name, age), gpa(gpa) {}
    Student(const Student& other)                  : Student(other.name, other.age, other.gpa) {}
    Student& operator=(const Student& other)
    { Person::operator=(other); gpa = other.gpa; return *this; }

    friend std::ostream& operator<<(std::ostream& os, const Student& rhs) {...}
};
```

godbolt.org/z/vheTeEdvW

B2B: Function Overloading – pass me a slice of that

```
struct Person
{
    Person(std::string name, int age) : name(name), age(age) {}
    Person(const Person& other) : Person(other.name, other.age) {}
    Person& operator=(const Person& other) {...}
};
struct Student : public Person
{
    Student(std::string name, int age, float gpa) : Person(name, age), gpa(gpa) {}
    Student(const Student& other) : Student(other.name, other.age, other.gpa) {}
    Student& operator=(const Student& other) {...}
};
int main()
{
    Person person ("bob", 14);
    Student student("sally", 16, 3.5);
    std::cout << "person:  " << person << '\n';
    std::cout << "student: " << student << '\n';
    Person slice = student;
    std::cout << "person/slice of student : " << slice << '\n';
}
```

Output:

person: bob 14

student: sally 16 3.5

person/slice of student: sally 16

godbolt.org/z/vheTeEdvW

B2B: Function Overloading – moving & constructing



B2B: Function Overloading – moving forward, moving back

```
struct DArray
{
    std::shared_ptr<int[]> data;
    DArray(size_t size) : data(std::make_shared<int[]>(size)) {}
    DArray(DArray&& other) noexcept : data(other.data) { other.data = nullptr; }
    DArray& operator=(DArray&& other) noexcept
    {
        if (this != &other)
        {
            data = other.data;
            other.data = nullptr;
        }
        return *this; }
};

int main()
{
    DArray arr1(10);
    DArray arr2(std::move(arr1));
    arr1 = std::move(arr2);

    std::cout << "arr1: " << arr1 << '\n';
    std::cout << "arr1: " << arr1 << '\n';
    std::cout << "arr2: " << arr2 << '\n';
    std::cout << "arr1: " << arr1 << '\n';
    std::cout << "arr2: " << arr2 << '\n';
}
```

Output:

```
arr1: 10 0x319ce2b0
arr1: 0 0
arr2: 10 0x319ce2b0
arr1: 10 0x319ce2b0
arr2: 0 0
```

godbolt.org/z/TEYWKe3Yf

B2B: Function Overloading – cut me a slice of that

```
struct DArray
{
    size_t size;  std::shared_ptr<int[]> data;
    DArray(size_t size) : size(size),data(std::make_shared<int[]>(size)) {}
    DArray(DArray&& other) noexcept : size(other.size), data(other.data) {...}
    DArray& operator=(DArray&& other) noexcept {...}
};

struct MyArray : public DArray
{
    std::string name;
    MyArray(std::string name, size_t size) : DArray(size), name(name) {}
    MyArray(MyArray&& other) noexcept : DArray(std::move(other)),name(other.name){...}
    MyArray& operator=(MyArray&& other) noexcept
        {... DArray::operator=(std::move(other)); ...}
};

int main()
{
    MyArray myArray("fred", 5);
    DArray dArray(std::move(myArray));
    std::cout << "myArray: " << myArray << '\n';
    std::cout << "dArray:  " << dArray  << '\n';
    std::cout << "myArray: " << myArray << '\n';
}
```

Output:

myArray: fred 5 0x3c5a52b0

dArray: 5 0x3c5a52b0

myArray: fred 0 0

godbolt.org/z/n8bPfccTE

B2B: Function Overloading – movement for free

Almost all standard library objects (classes and containers) introduced or updated in C++11 and later have built-in move constructors and move assignment operators.

- **Key Standard Library Types with Move Operations**
- **Standard Containers:** [`std::vector`](#), [`std::string`](#), [`std::list`](#), [`std::map`](#), [`std::unordered\_map`](#), [`std::set`](#), and `std::deque`.
- **Smart Pointers:** [`std::unique\_ptr`](#) (which *only* supports moving, copying is deleted) and [`std::shared\_ptr`](#).
- **I/O and Streams:** Most stream types do *not* support moving or copying, but stream-adjacent or utility types like [`std::stringstream`](#) support move operations in modern C++.
- **Thread Support & Synchronization:** Types like [`std::thread`](#) and [`std::future`](#) are movable (transferring ownership of the underlying thread or state) but not copyable.

B2B: Function Overloading – friends get special access



B2B: Function Overloading – private member functions

```
class Bar
{
private:
    void Foo()                { std::print("Bar::Foo()      \n"); }
    void Foo(int i)           { std::print("Bar::Foo({})    \n", i); }
    void Foo(double d)        { std::print("Bar::Foo({})    \n", d); }
    void Foo(std::string s)    { std::print("Bar::Foo(\"{} \") \n", s); }
};

int main()
{
    Bar bar;
    bar.Foo();                // error: 'void Bar::Foo()' is private within this context
    bar.Foo(1);               // error: 'void Bar::Foo(int)' is private within this context
    bar.Foo(2.3);             // error: 'void Bar::Foo(double)' is private within this context
    bar.Foo("hello");         // error: 'void Bar::Foo(std::string)' is private within this context
}
```

godbolt.org/z/4YW7WEEPo

B2B: Function Overloading – global free function accessors

```
class Bar
{
private:
    void Foo()                { std::print("Bar::Foo()      \n"); }
    void Foo(int i)           { std::print("Bar::Foo({})    \n", i); }
    void Foo(double d)        { std::print("Bar::Foo({})    \n", d); }
    void Foo(std::string s)    { std::print("Bar::Foo(\"{} \") \n", s); }
};

void Qaz(Bar &bar)            { bar.Foo(); } // error: private within this context
void Qaz(Bar &bar, int i)     { bar.Foo(i); } // error: private within this context
void Qaz(Bar &bar, double d)  { bar.Foo(d); } // error: private within this context
void Qaz(Bar &bar, std::string s) { bar.Foo(s); } // error: private within this context

int main()
{
    Bar bar;
    Qaz(bar);
    Qaz(bar, 1);
    Qaz(bar, 2.3);
    Qaz(bar, "hello");
}
```

godbolt.org/z/K7n7qvEd

B2B: Function Overloading – hidden friends and ADL

```
class Bar
{
private:
    void Foo()                { std::print("Bar::Foo()      \n"); }
    void Foo(int i)           { std::print("Bar::Foo({})    \n", i); }
    void Foo(double d)        { std::print("Bar::Foo({})    \n", d); }
    void Foo(std::string s)    { std::print("Bar::Foo(\"{} \") \n", s); }
    //public:
    friend void Qaz(Bar &bar)   { bar.Foo(); }
    friend void Qaz(Bar &bar, int i) { bar.Foo(i); }
    friend void Qaz(Bar &bar, double d) { bar.Foo(d); }
    friend void Qaz(Bar &bar, std::string s) { bar.Foo(s); }
};

int main()
{
    Bar bar;
    Qaz(bar);
    Qaz(bar, 1);
    Qaz(bar, 2.3);
    Qaz(bar, "hello");
}
```

Output:
Bar:: **Foo**()
Bar:: **Foo**(1)
Bar:: **Foo**(2.3)
Bar:: **Foo**("hello")

godbolt.org/z/eY8xPvrE

B2B: Function Overloading – questions ?



B2B: Function Overloading – operator overloading



B2B: Function Overloading – operator overloading

Unary Operators: (one argument) **Binary Operators:** require LHS & RHS

+ (Unary plus)
- (Unary minus)
! (Logical NOT)
~ (Bitwise NOT)
++ (Prefix and Postfix Increment)
-- (Prefix and Postfix Decrement)

User-defined conversion function / aka
Casting operator (0 args, must be a non-static member)

We are not going to talking about overloading:

& (Address-of)
* (Indirection / Dereference)
-> (Member selection / Arrow operator)
->* (Pointer-to-member selection)
, (Comma operator)

Arithmetic Operators: +, -, *, /, %

Logical Operators: &&, ||

Bitwise Operators: &, |, ^, <<, >>

Stream Operators: <<, >>

Assignment Operators:

Basic Assignment: =

Arithmetic Assignment: -=, *=, /=, &=

Bitwise Assignment : &=, |=, ^=, <<=, >>=

Relational Operators: <, >, <=, >=, ==, !=, <=>

Subscript Operators: [], [...]

B2B: Operator Overloading – smooth operators

Arithmetic	Assignment	Inc / dec	Comparison	Member access
Arithmetic: +a -a a + b a - b a * b a / b a % b	a = b Arithmetic: a += b a -= b a *= b a /= b a %= b	++a a++ --a a--	Relational: a == b a != b a < b a > b a <= b a >= b	Dereference: *a &a a->b a->*b
Logical: (bitwise) ~a a & b a b a ^ b	Logical: (bitwise) a &= b a = b a ^= b	Logical: (boolean) !a a && b a b	Spaceship: a <=> b	Subscript: a[] a[,,,]
Shift: (bitwise) a << b a >> b	Shift: (bitwise) a <<= b a >>= b	Stream: a << b a >> b		Non-overloadable: a.b a.*b ?:

B2B: Operator Overloading – operator rules

Like Function overloading Operator overloading has its own set of rules.

- **Operators** are just **functions** with **special names**,
- Can not change the number of **expected arguments** ,
 - But you can change the **types of parameters**.
- Can not change the **Order of Precedence of Operators**,
- Can not create new operators,
- Can not **change built-in meaning for primitive** types,
- Do not break **logical intuition**: + should do **addition**, – should do **subtraction**, ...
 - **Principle of Least Astonishment (POLA)**
- **Short-circuit evaluation** is lost for overloaded **&& and ||**,
- Comma operator sequencing: **strips away** its native strict **left-to-right evaluation** order guarantee,

B2B: Operator Overloading – operator overloading

```
struct Object { //could be any type of members
    Object operator-() const;
    Object operator+(const Counter& Object) const;
    Object operator++(int) const; //postfix++
};
Int main()
{
    Object o1{1};
    Object o2{2};

    // When a compiler Sees (for non-intrinsic types):
    Object o3 = o1 + o2;
    Object o4 = -o3;
    Object o5 = o4++;

    // the compiler internally reads it as and looks for the matching function.
    Object o3 = o1.operator+(o2);           // or operator+(o1,o2)    or friends
    Object o4 = o3.operator-();              // or operator-(o3)      or friends
    Object o5 = o4.operator++(int);          // or operator++(o4,int) or friends
}
```

B2B: Operator Overloading – precedence in order



B2B: Operator Overloading – order of precedence

Precedence	Operator Category	Description / Examples	Associativity
1 (Highest)	Postfix / Scope	:: () [] . -> ++ -- (postfix)	Left-to-right
2	Unary	+ - ! ~ ++ -- (prefix) *	Right-to-left
3	Multiplicative	* / %	Left-to-right
4	Additive	+ -	Left-to-right
5	Shift / Stream	<< >>	Left-to-right
6	Three-Way Comparison	<=> (Spaceship Operator)	Left-to-right
7	Relational	< <= > >=	Left-to-right
8	Equality	== !=	Left-to-right
9	Bitwise AND	&	Left-to-right
10	Bitwise XOR / OR	^	Left-to-right
11	Logical AND	&&	Left-to-right
12	Logical OR		Left-to-right
13	Ternary / Conditional	? :	Right-to-left
14	Assignment	= += -= *= /= %= <<= >>= &= ^= =	Right-to-left
15 (Lowest)	Comma	,	Left-to-right

B2B: Operator Overloading – one is the loneliest number



B2B: Operator Overloading – unary operators

Arithmetic & Logical (+, -, !, ~) - Free Function:

```
struct MyInt
{
    int x{};
};

MyInt operator+(const MyInt& obj) {return MyInt(+obj.x);}; //positive(like a no-op)
MyInt operator-(const MyInt& obj) {return MyInt(-obj.x);}; //negate(make negative)
MyInt operator!(const MyInt& obj) {return MyInt(!obj.x);}; //logical not
MyInt operator~(const MyInt& obj) {return MyInt(~obj.x);}; //bitwise not

MyInt int1{1};
MyInt int2 = -int1;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << <'\\n';
```

Output:
Int.x1: 1 int2.x: -1

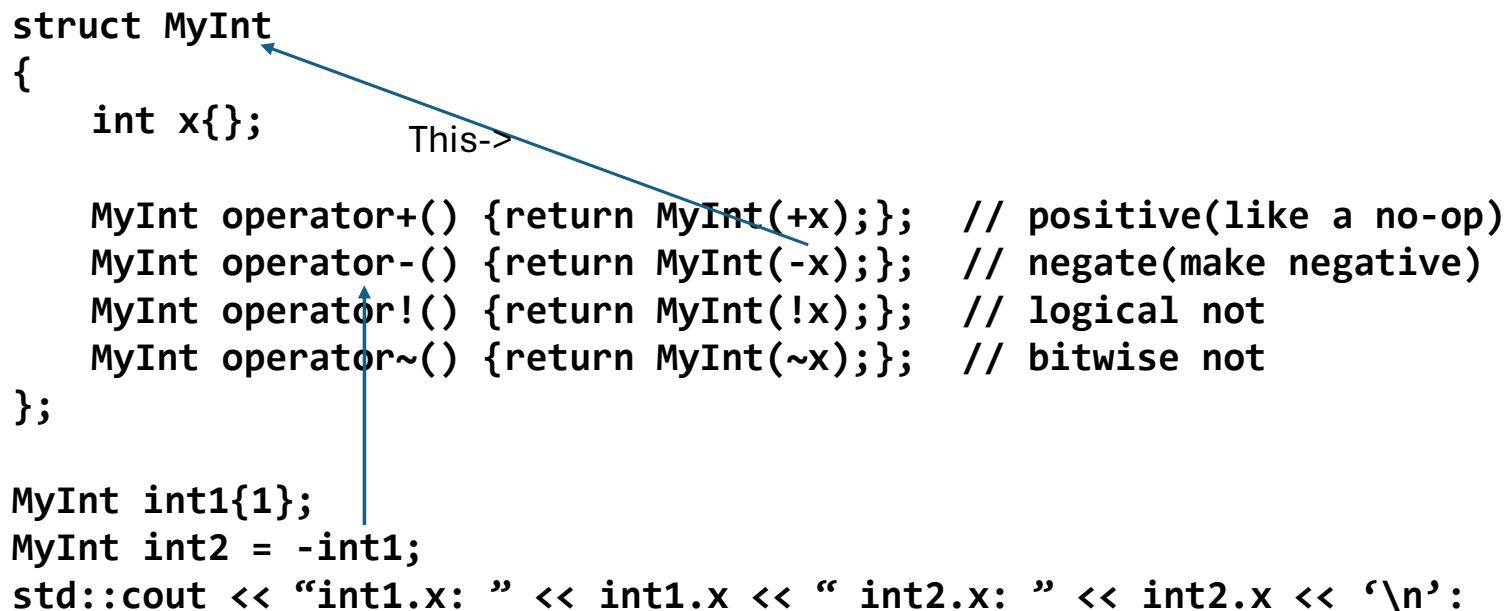
B2B: Operator Overloading – unary operators

Arithmetic & Logical (+, -, !, ~) - Member Function:

```
struct MyInt
{
    int x{};

    MyInt operator+() {return MyInt(+x);}; // positive(like a no-op)
    MyInt operator-() {return MyInt(-x);}; // negate(make negative)
    MyInt operator!() {return MyInt(!x);}; // logical not
    MyInt operator~() {return MyInt(~x);}; // bitwise not
};

MyInt int1{1};
MyInt int2 = -int1;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n':
```



Output:

Int.x1: 1 int2.x: -1

B2B: Operator Overloading – unary operators

Arithmetic & Logical (+, -, !, ~) - Friend Function:

```
struct MyInt
{
    int x{};

    friend MyInt operator+(const MyInt& obj){return MyInt(+obj.x);} //positive/no-op
    friend MyInt operator-(const MyInt& obj){return MyInt(-obj.x);} //Negate
    friend MyInt operator!(const MyInt& obj){return MyInt(!obj.x);} //logical not
    friend MyInt operator~(const MyInt& obj){return MyInt(~obj.x);} //bitwise not
};

MyInt int1{1};
MyInt int2 = -int1;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n';
```

Output:
Int.x1: 1 int2.x: -1

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, Free Function:

```
struct Counter
```

```
{  
    int value{};  
};
```

```
Counter& operator++(Counter& counter)
```

```
//Prefix Operators ++x, --y
```

```
{  
    rhs.value++;  
    return rhs;  
}
```

```
// Modify object
```

```
// Return the updated object
```

```
Counter operator--(Counter& counter, int)
```

```
// Postfix Operators x++, y--
```

```
{  
    // the extra unused param is to differentiate it from a prefix
```

```
    Counter temp(counter);
```

```
    counter.value--;
```

```
// Modify object
```

```
    return temp;
```

```
// Return a copy of the original state
```

```
}
```

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, Free Function:

```
struct MyInt
{
    int x{};
};

MyInt& operator--(MyInt& myInt) {...; return myInt;} // prefix --X
MyInt& operator++(MyInt& myInt) {...; return myInt;}

MyInt  operator--(MyInt& myInt, int) {MyInt temp(myInt); ...; return temp;} // postfix
MyInt  operator++(MyInt& myInt, int) {MyInt temp(myInt); ...; return temp;} // x++

MyInt int1{2}, int2{5};
++int1; int2--;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n';
```

Output:
int1.x: 3 int2.x: 4

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, Member Function:

```
struct Counter
{
    int value{};

    Counter& operator++()                //Prefix Operators ++x, --y
    {
        c.value++;
        return c;
        // Modify object
        // Return the updated object
    }
    Counter operator--(int)              // Postfix Operators x++, y-
    {                                     // the extra unused param is to differentiate it from a prefix
        Counter temp = c;
        c.value--;
        return temp;
        // Modify object
        // Return a copy of the original state
    }
};
```

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, member Function:

```
struct MyInt
{
    int x{}; This-> This->

    //prefix, return after value
    MyInt& operator--() {...; return *this;}
    MyInt& operator++() {...; return *this;}

    //postfix, return before value
    MyInt operator--(int) {MyInt temp(*this); ...; return temp;}
    MyInt operator++(int) {MyInt temp(*this); ...; return temp;}
};

MyInt int1{2},int2{5};
++int1; int2--;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n';
```

Output:

```
Int.x1: 3 int2.x: 4
```

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, Hidden Friends:

```
struct Counter
{
    int value{};

    friend Counter& operator++(Counter& c)    //Prefix Operators ++x, --y
    {
        c.value--;                          // Modify object
        return c;                          // Return the updated object
    }
    friend Counter operator--(Counter& c, int) // Postfix Operators x++, y-
    {                                         // the extra unused param is to differentiate it from a prefix
        Counter temp = c;
        c.value--;                          // Modify object
        return temp;                        // Return copy original state
    }
};
```

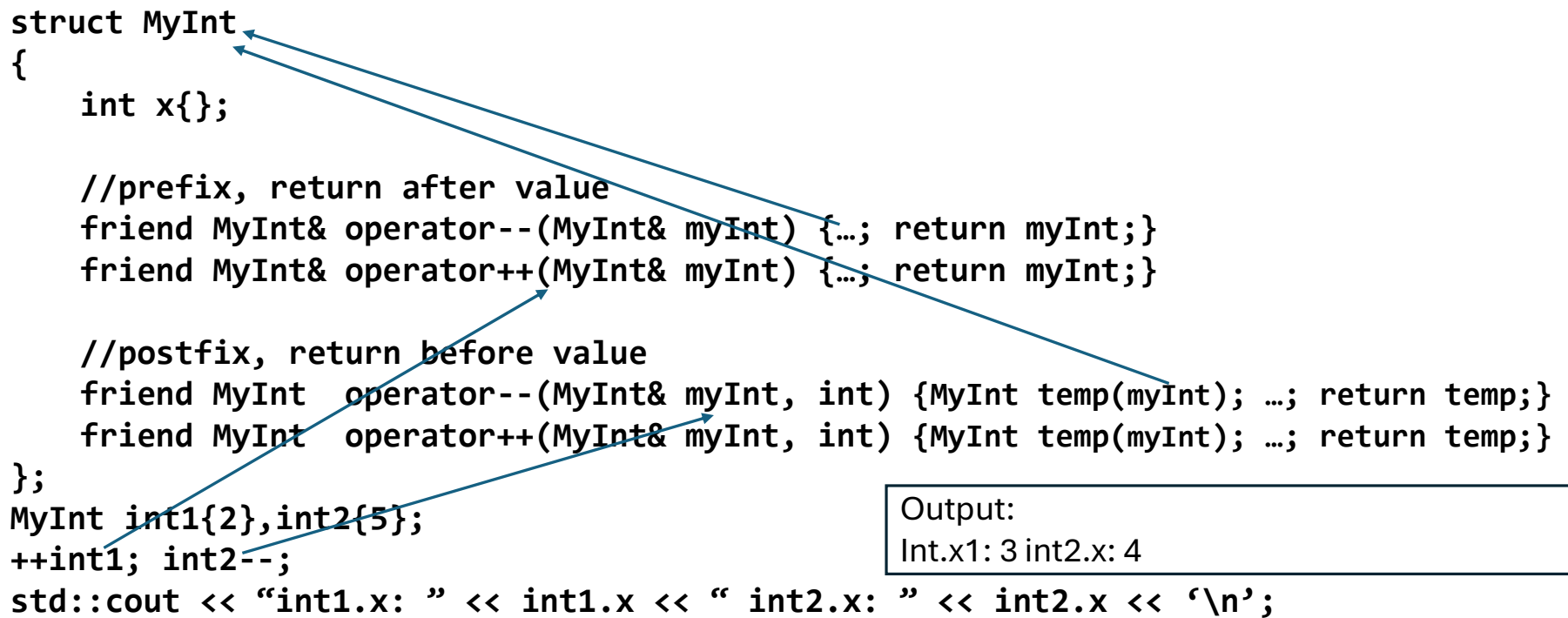
B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, friend Function:

```
struct MyInt
{
    int x{};

    //prefix, return after value
    friend MyInt& operator--(MyInt& myInt) {...; return myInt;}
    friend MyInt& operator++(MyInt& myInt) {...; return myInt;}

    //postfix, return before value
    friend MyInt operator--(MyInt& myInt, int) {MyInt temp(myInt); ...; return temp;}
    friend MyInt operator++(MyInt& myInt, int) {MyInt temp(myInt); ...; return temp;}
};
MyInt int1{2},int2{5};
++int1; int2--;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n';
```



Output:
Int.x1: 3 int2.x: 4

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, Hidden Friends: (hybrid/D.R.Y. style)

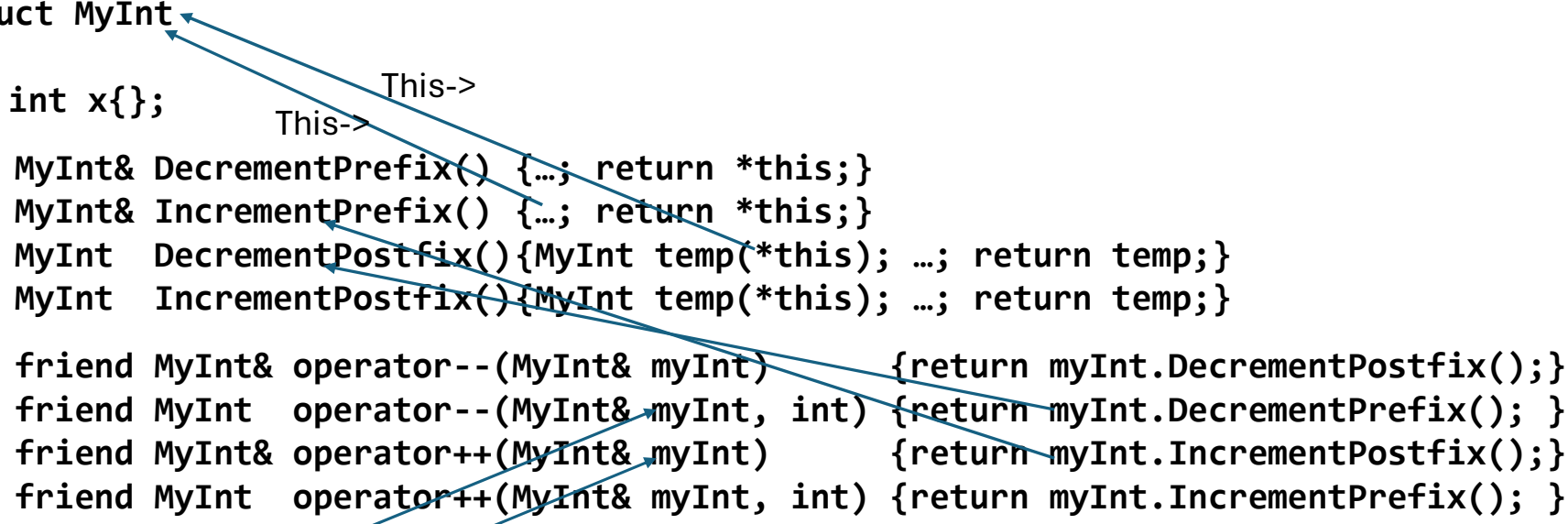
```
struct Counter
{
    int value{};
    Counter& DecrementPrefix()
    {
        value--;
        return *this;    // Modify, return after
    }
    Counter DecrementPostfix()
    {
        Counter temp = c;
        value--;
        return temp;    // Modify, return before
    }
    friend Counter& operator--(Counter& counter)    //Prefix Operators ++x, --y
    { return counter.DecrementPrefix(); }
    friend Counter operator--(Counter& counter, int) //Postfix Operators x++, y--
    { return counter.DecrementPostfix(); }
};
```

B2B: Operator Overloading – unary operators

Prefix ++x, Postfix y--, friend Function: (hybrid/D.R.Y. style)

```
struct MyInt
{
    int x{};
    MyInt& DecrementPrefix() {...; return *this;}
    MyInt& IncrementPrefix() {...; return *this;}
    MyInt DecrementPostfix(){MyInt temp(*this); ...; return temp;}
    MyInt IncrementPostfix(){MyInt temp(*this); ...; return temp;}

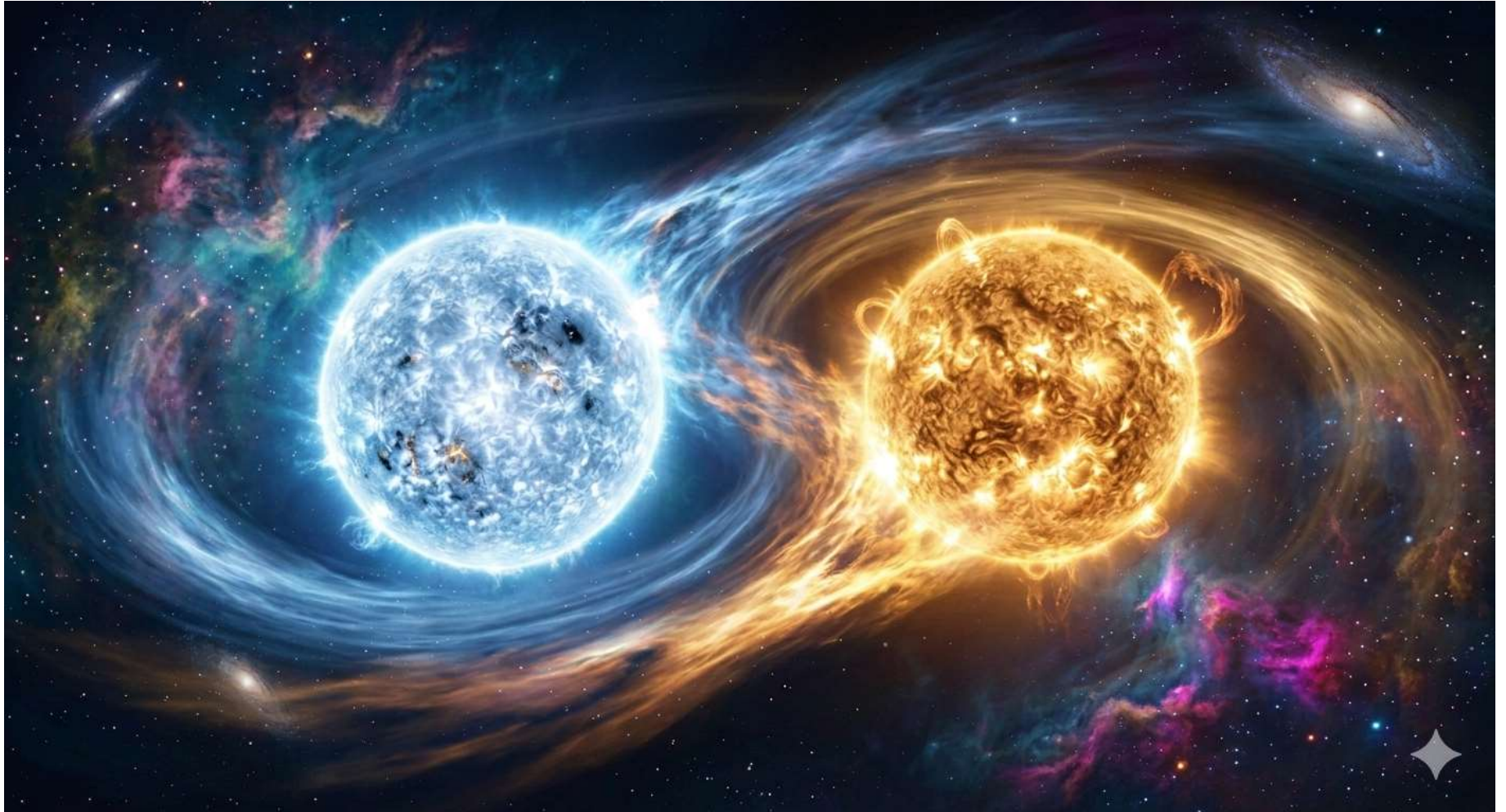
    friend MyInt& operator--(MyInt& myInt) {return myInt.DecrementPostfix();}
    friend MyInt operator--(MyInt& myInt, int) {return myInt.DecrementPrefix(); }
    friend MyInt& operator++(MyInt& myInt) {return myInt.IncrementPostfix();}
    friend MyInt operator++(MyInt& myInt, int) {return myInt.IncrementPrefix(); }
};
```



```
MyInt int1{2},int2{5};
Int1--; ++int2;
std::cout << "int1.x: " << int1.x << " int2.x: " << int2.x << '\n';
```

Output:
Int.x1: 3 int2.x: 4

B2B: Function Overloading – two can be as bad as one



B2B: Operator Overloading – order in chaos

grouping operators in pairs of runs

```
x = q + w + e + r + t + y;
```

```
x = (((((q + w) + e) + r) + t) + y);
```



```
//using temp variables
```

```
t1= operator+(q, w);
```

```
t2= operator+(t1, e);
```

```
t3= operator+(t2, r);
```

```
t4= operator+(t3, t);
```

```
x = operator+(t4, y);
```

```
//In nested format
```

```
x = operator+(operator+(operator+(operator+(operator+(q, w), e), r), t), y);
```

B2B: Operator Overloading – chaos in order

grouping operators in pairs by precedence

```
x = q + w * e - r / t + y;  
x = q + (w * e) - (r / t) + y;  
x = (((q + (w * e)) - (r / t)) + y);
```



//using temp variables

```
t1= operator*(w, e);  
t2= operator+(q, t1);  
t3= operator/(r, t);  
t4= operator-(t2, t3);  
x = operator+(t4, y);
```

//In nested format

```
x = operator+(operator-(operator+(q, operator*(w, e)), operator/(r, t)), y);
```

B2B: Operator Overloading – binary operators

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Free functions:

```
struct Counter
{
    int value{};
};

Counter operator+(const Counter& lhs, const Counter& rhs)
{
    return Counter(lhs.value + rhs.value);
}

Counter operator*(const Counter& lhs, const Counter& rhs)
{
    return Counter(lhs.value * rhs.value);
}
```

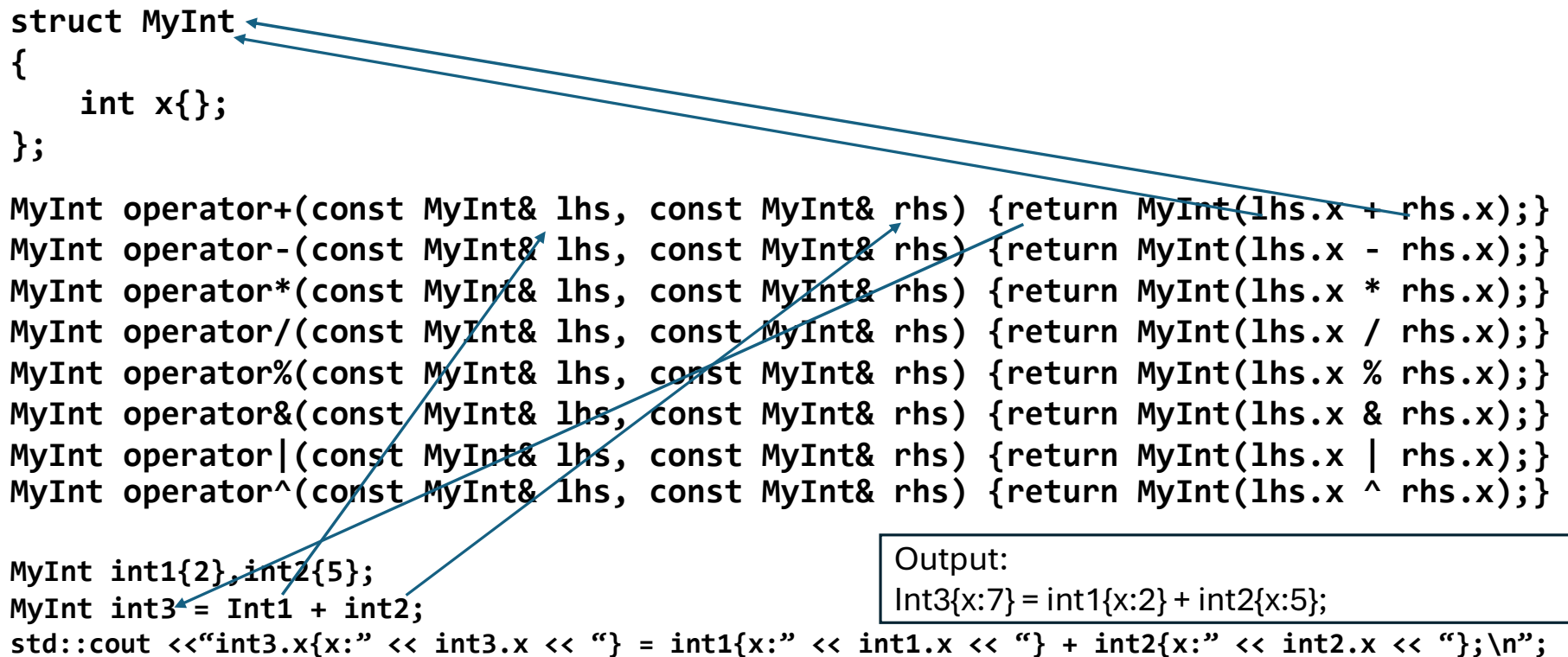
B2B: Operator Overloading –

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Free functions:

```
struct MyInt
{
    int x{};
};

MyInt operator+(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x + rhs.x);}
MyInt operator-(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x - rhs.x);}
MyInt operator*(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x * rhs.x);}
MyInt operator/(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x / rhs.x);}
MyInt operator%(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x % rhs.x);}
MyInt operator&(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x & rhs.x);}
MyInt operator|(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x | rhs.x);}
MyInt operator^(const MyInt& lhs, const MyInt& rhs) {return MyInt(lhs.x ^ rhs.x);}

MyInt int1{2}, int2{5};
MyInt int3 = int1 + int2;
std::cout << "int3.x{x:}" << int3.x << " } = int1{x:}" << int1.x << " } + int2{x:}" << int2.x << " };\n";
```



Output:
int3{x:7} = int1{x:2} + int2{x:5};

B2B: Operator Overloading –

Arithmetic & Logical +=, -=, *=, /=, %=, &=, |=, ^= – Free functions:

```
struct MyInt  
{  
    int x{};  
};
```

```
MyInt& operator+=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x + lhs.x);}  
MyInt& operator-=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x - lhs.x);}  
MyInt& operator*=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x * lhs.x);}  
MyInt& operator/=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x / lhs.x);}  
MyInt& operator%=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x % lhs.x);}  
MyInt& operator&=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x & lhs.x);}  
MyInt& operator|=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x | lhs.x);}  
MyInt& operator^=(MyInt& lhs, const MyInt& rhs) {return lhs=MyInt(rhs.x ^ lhs.x);}
```

```
MyInt int1{2},int2{5};  
Int1 += int2;  
std::cout <<"int1{" << int1.x << "}" += int2{" << int2.x << "};\n";
```

//before int1{2}

Output:

Int1{7} += int2{5};

B2B: Operator Overloading – binary operators

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Member functions:

```
struct Counter
{
    int value{};

    Counter operator+(const Counter& lhs, const Counter& rhs) const
    {
        return Counter(lhs.value + rhs.value);
    }

    Counter operator*(const Counter& lhs, const Counter& rhs) const
    {
        return Counter(lhs.value * rhs.value);
    }
};
```

B2B: Operator Overloading –

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Member functions:

```
struct MyInt
{
    int x{};

    MyInt operator+(const MyInt& rhs) const {return MyInt(x + rhs.x);}
    MyInt operator-(const MyInt& rhs) const {return MyInt(x - rhs.x);}
    MyInt operator*(const MyInt& rhs) const {return MyInt(x * rhs.x);}
    MyInt operator/(const MyInt& rhs) const {return MyInt(x / rhs.x);}
    MyInt operator%(const MyInt& rhs) const {return MyInt(x % rhs.x);}
    MyInt operator&(const MyInt& rhs) const {return MyInt(x & rhs.x);} //bitwise
    MyInt operator|(const MyInt& rhs) const {return MyInt(x | rhs.x);} //bitwise
    MyInt operator^(const MyInt& rhs) const {return MyInt(x ^ rhs.x);} //bitwise
};

MyInt int1{2}, int2{5};
MyInt int3 = int1 + int2;
std::cout << "int3.x{x:}" << int3.x << " } = int1{x:}" << int1.x << " } + int2{x:}" << int2.x << " };\n";
```

This->

Output:
int3{x:7} = int1{x:2} + int2{x:5};

B2B: Operator Overloading –

Arithmetic & Logical +=, -=, *=, /=, %=, &=, |=, ^= – Member functions:

```
struct MyInt
{
    int x{};

    MyInt& operator+=(const MyInt& rhs) {x += rhs.x; return *this;}
    MyInt& operator-=(const MyInt& rhs) {x -= rhs.x; return *this;}
    MyInt& operator*=(const MyInt& rhs) {x *= rhs.x; return *this;}
    MyInt& operator/=(const MyInt& rhs) {x /= rhs.x; return *this;}
    MyInt& operator%=(const MyInt& rhs) {x %= rhs.x; return *this;}
    MyInt& operator&=(const MyInt& rhs) {x &= rhs.x; return *this;} //bitwise
    MyInt& operator|=(const MyInt& rhs) {x |= rhs.x; return *this;} //bitwise
    MyInt& operator^=(const MyInt& rhs) {x ^= rhs.x; return *this;} //bitwise
};

MyInt int1{2},int2{5};
int1 += int2;
std::cout <<"int1{" << int1.x << "} += int2{" << int2.x << "};\n";
```

This->

//before int1{2}
Output:
int1{7} += int2{5};

B2B: Operator Overloading – binary operators

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Hidden Friends: (D.R.Y. style)

```
struct Count
{
    int value{};

    Counter& Add      (const Count& rhs)  { value + rhs.value; return *this; }
    Counter& Multiply(const Count& rhs)  { value * rhs.value; return *this; }

    friend Count operator+ (const Count& lhs, const Count& rhs)  {Count t(lhs); return  t.Add(rhs);    }
    friend& Count operator+=(      Count& lhs, const Count& rhs)  {      return lhs.Add(rhs);    }

    friend Count operator* (const Count & lhs, const Count & rhs) {Count t(lhs); return  t.Multiply(rhs);}
    friend& Count operator*=(      Count & lhs, const Count & rhs) {      return lhs.Multiply(rhs);}
};
```

B2B: Operator Overloading –

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Hidden Friends: (D.R.Y. style)

```
struct MyInt {  
    int x{};  
    This->  
    MyInt Add (const MyInt& rhs) const {return MyInt(x + rhs.x);}   
    MyInt Sub (const MyInt& rhs) const {return MyInt(x - rhs.x);}   
    MyInt Mult(const MyInt& rhs) const {return MyInt(x * rhs.x);}   
    MyInt Div (const MyInt& rhs) const {return MyInt(x / rhs.x);}   
    MyInt Mod (const MyInt& rhs) const {return MyInt(x % rhs.x);}   
    MyInt AND (const MyInt& rhs) const {return MyInt(x & rhs.x);} //bitwise AND   
    MyInt OR  (const MyInt& rhs) const {return MyInt(x | rhs.x);} //bitwise OR   
    MyInt XOR (const MyInt& rhs) const {return MyInt(x ^ rhs.x);} //bitwise XOR   
  
    friend MyInt operator+(const MyInt& lhs, const MyInt& rhs) {return lhs.Add (rhs);}   
    friend MyInt operator-(const MyInt& lhs, const MyInt& rhs) {return lhs.Sub (rhs);}   
    friend MyInt operator*(const MyInt& lhs, const MyInt& rhs) {return lhs.Mult(rhs);}   
    friend MyInt operator/(const MyInt& lhs, const MyInt& rhs) {return lhs.Div (rhs);}   
    friend MyInt operator%(const MyInt& lhs, const MyInt& rhs) {return lhs.Mod (rhs);}   
    friend MyInt operator&(const MyInt& lhs, const MyInt& rhs) {return lhs.AND (rhs);}   
    friend MyInt operator|(const MyInt& lhs, const MyInt& rhs) {return lhs.OR  (rhs);}   
    friend MyInt operator^(const MyInt& lhs, const MyInt& rhs) {return lhs.XOR (rhs);}   
};  
MyInt int3 = MyInt{2} + MyInt{5};
```

B2B: Operator Overloading –

Arithmetic & Logical +, -, *, /, %, &, |, ^ – Hidden Friends: (D.R.Y. style)

```
struct MyInt {  
    int x{};  
  
    MyInt Add (const MyInt& rhs) const {return MyInt(x + rhs.x);}   
    MyInt Sub (const MyInt& rhs) const {return MyInt(x - rhs.x);}   
    MyInt Mult(const MyInt& rhs) const {return MyInt(x * rhs.x);}   
    MyInt Div (const MyInt& rhs) const {return MyInt(x / rhs.x);}   
    MyInt Mod (const MyInt& rhs) const {return MyInt(x % rhs.x);}   
    MyInt AND (const MyInt& rhs) const {return MyInt(x & rhs.x);} //bitwise AND   
    MyInt OR  (const MyInt& rhs) const {return MyInt(x | rhs.x);} //bitwise OR   
    MyInt XOR (const MyInt& rhs) const {return MyInt(x ^ rhs.x);} //bitwise XOR   
  
    friend MyInt& operator+(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.Add (rhs);}   
    friend MyInt& operator-(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.Sub (rhs);}   
    friend MyInt& operator*(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.Mult(rhs);}   
    friend MyInt& operator/(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.Div (rhs);}   
    friend MyInt& operator%(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.Mod (rhs);}   
    friend MyInt& operator&(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.AND (rhs);}   
    friend MyInt& operator|(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.OR  (rhs);}   
    friend MyInt& operator^(MyInt& lhs, const MyInt& rhs) {return lhs = lhs.XOR (rhs);}   
};  
MyInt int1{2}; int1 += MyInt{5};
```

Diagram illustrating the implementation of operator overloading for the `MyInt` struct using the D.R.Y. (Don't Repeat Yourself) style. The struct `MyInt` contains a member variable `x` and several member functions for arithmetic and logical operations. The friend functions are used to overload the operators `+`, `-`, `*`, `/`, `%`, `&`, `|`, and `^`. The code is annotated with blue arrows showing the mapping from the struct to the functions. A red box highlights the `const` keyword in the member functions and the first parameter of the friend functions.

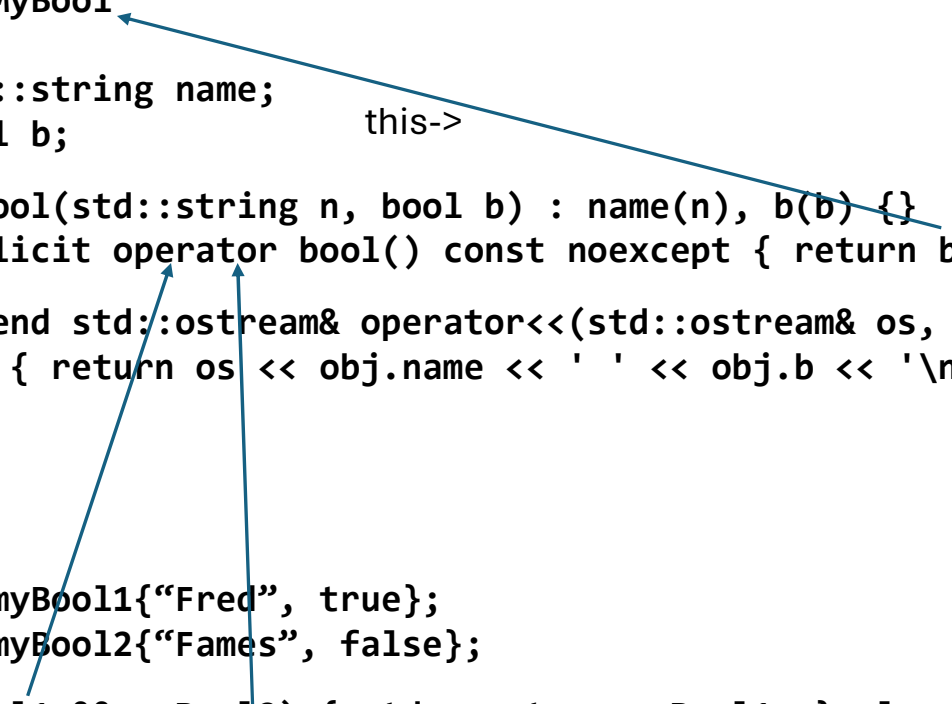
B2B: Operator Overloading – it is only logical



B2B: Operator Overloading – shape-shifting & disguises

Boolean operators &&, || – no overloads/typecast operator:

```
struct MyBool
{
    std::string name;
    bool b;
    MyBool(std::string n, bool b) : name(n), b(b) {}
    explicit operator bool() const noexcept { return b; };
    friend std::ostream& operator<<(std::ostream& os, const MyBool& obj)
    { return os << obj.name << ' ' << obj.b << '\n'; }
};
```



```
MyBool myBool1{"Fred", true};
MyBool myBool2{"James", false};
```

```
If(myBool1 && myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
If(myBool1 || myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
```

Output:
James 0
Fred 1

B2B: Operator Overloading –

Boolean operators &&, || – Member functions:

```
struct MyBool
{
    std::string name;
    bool b;
    MyBool(std::string n, bool b) : name(n), b(b) {}
    explicit operator bool() const noexcept { return b; };

    friend std::ostream& operator<<(std::ostream& os, const MyBool& obj)
    { return os << obj.name << ' ' << obj.b << '\n'; }

    bool operator&&(const MyBool& rhs) { return b && rhs.b; }
    bool operator||(const MyBool& rhs) { return b || rhs.b; }
};

MyBool myBool1{"Fred", true};
MyBool myBool2{"James", false};

If(myBool1 && myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
If(myBool1 || myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
```

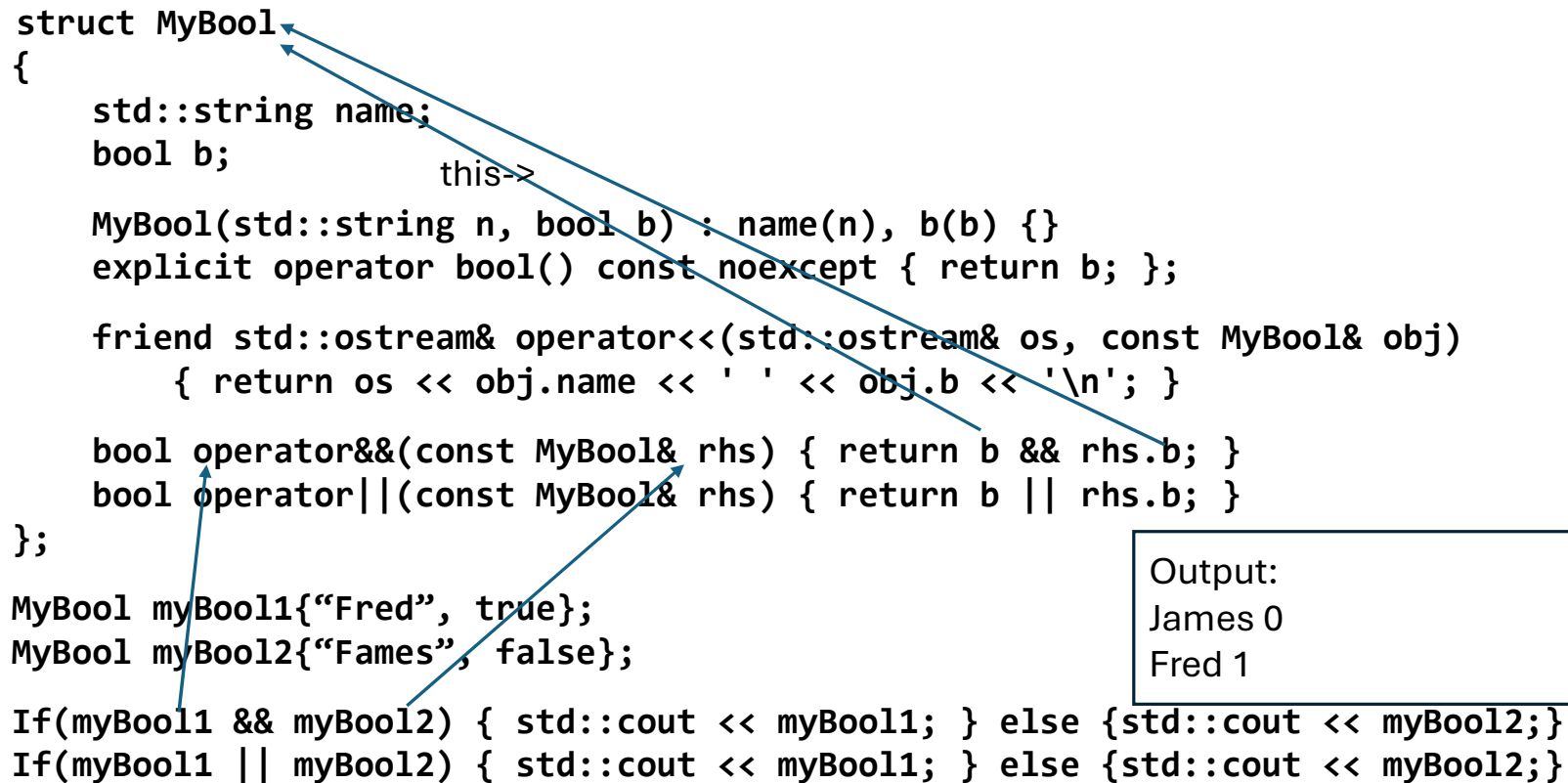


Diagram illustrating the relationship between the `MyBool` struct and the output:

- An arrow points from the `MyBool` struct definition to the output box.
- An arrow points from the `myBool1` variable to the output box.
- An arrow points from the `myBool2` variable to the output box.
- An arrow points from the `operator&&` function to the output box.
- An arrow points from the `operator||` function to the output box.

Output:

James 0

Fred 1

B2B: Operator Overloading –

Boolean operators &&, || – hidden friend functions:

```
struct MyBool
{
    std::string name;
    bool b;

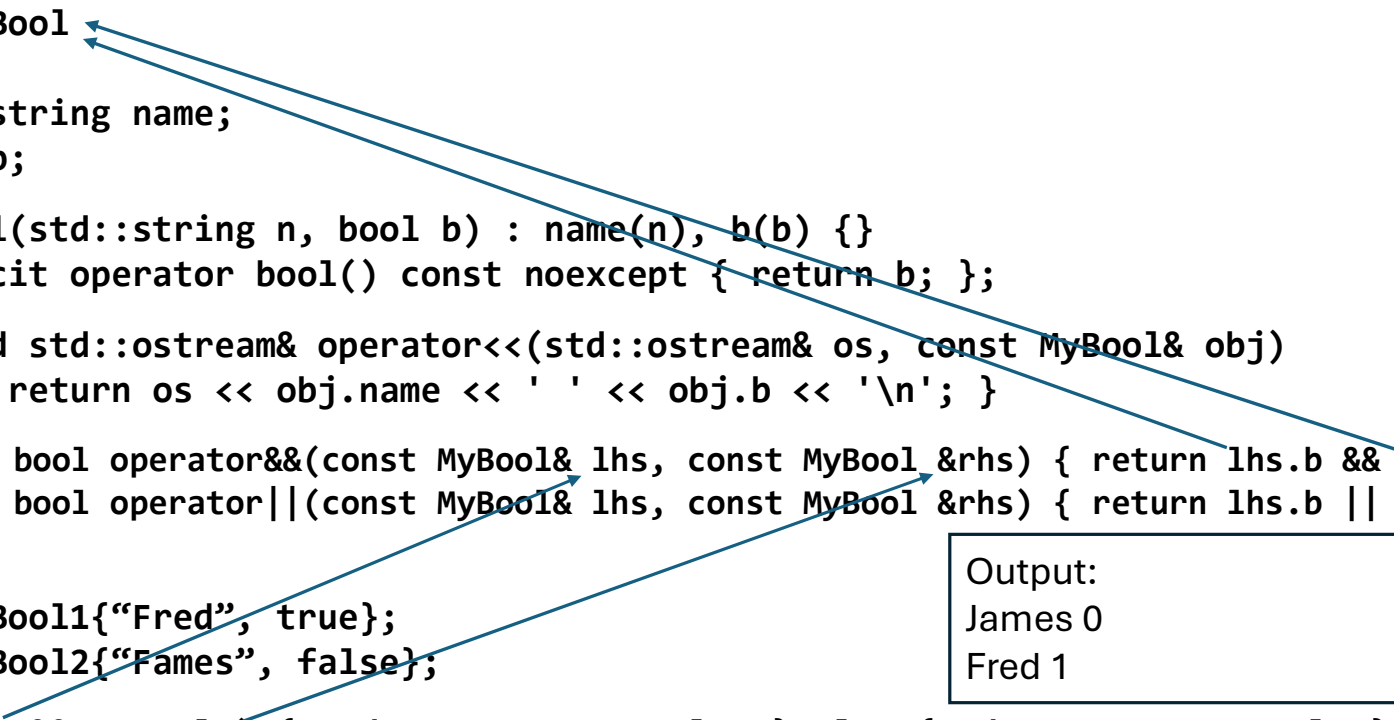
    MyBool(std::string n, bool b) : name(n), b(b) {}
    explicit operator bool() const noexcept { return b; };

    friend std::ostream& operator<<(std::ostream& os, const MyBool& obj)
        { return os << obj.name << ' ' << obj.b << '\n'; }

    friend bool operator&&(const MyBool& lhs, const MyBool& rhs) { return lhs.b && rhs.b; }
    friend bool operator||(const MyBool& lhs, const MyBool& rhs) { return lhs.b || rhs.b; }
};

MyBool myBool1{"Fred", true};
MyBool myBool2{"James", false};

If(myBool1 && myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
If(myBool1 || myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
```



Output:
James 0
Fred 1

B2B: Operator Overloading –

Boolean operators &&, || – hidden friend functions: (D.R.Y. style)

```
struct MyBool
{
    std::string name;
    bool b;           this->
    MyBool(std::string n, bool b) : name(n), b(b) {}
    explicit operator bool() const noexcept { return b; };

    friend std::ostream& operator<<(std::ostream& os, const MyBool& obj)
        { return os << obj.name << ' ' << obj.b << '\n'; }

    bool And(const MyBool &rhs) { return b && rhs.b; }
    bool Or (const MyBool &rhs) { return b || rhs.b; }

    friend bool operator&&(const MyBool& lhs, const MyBool &rhs) { return lhs.And(rhs); }
    friend bool operator||(const MyBool& lhs, const MyBool &rhs) { return lhs.Or (rhs); }
};

MyBool myBool1{"Fred", true};
MyBool myBool2{"Fames", false};
If(myBool1 && myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
If(myBool1 || myBool2) { std::cout << myBool1; } else {std::cout << myBool2;}
```

James 0
Fred 1

B2B: Operator Overloading – things are shaking (shifting)



B2B: Operator Overloading – <<, >> overloading

Classic bitwise shift operator

```
int main()
{
    uint32_t oldValue = 0x12345678;
    uint32_t newValue1 = oldValue << 4;
    uint32_t newValue2 = newValue1 >> 8;

    std::cout << "oldValue = 0x" << std::hex << oldValue << '\n';
    std::cout << "newValue1 = 0x" << std::hex << newValue1 << '\n';
    std::cout << "newValue2 = 0x" << std::hex << newValue2 << '\n';
}
```

Output:

```
oldValue = 0x12345678
newValue1 = 0x23456780
newValue2 = 0x00234567
```

godbolt.org/z/vKrca9xqM

B2B: Operator Overloading – <<, >> overloading

Free Functions

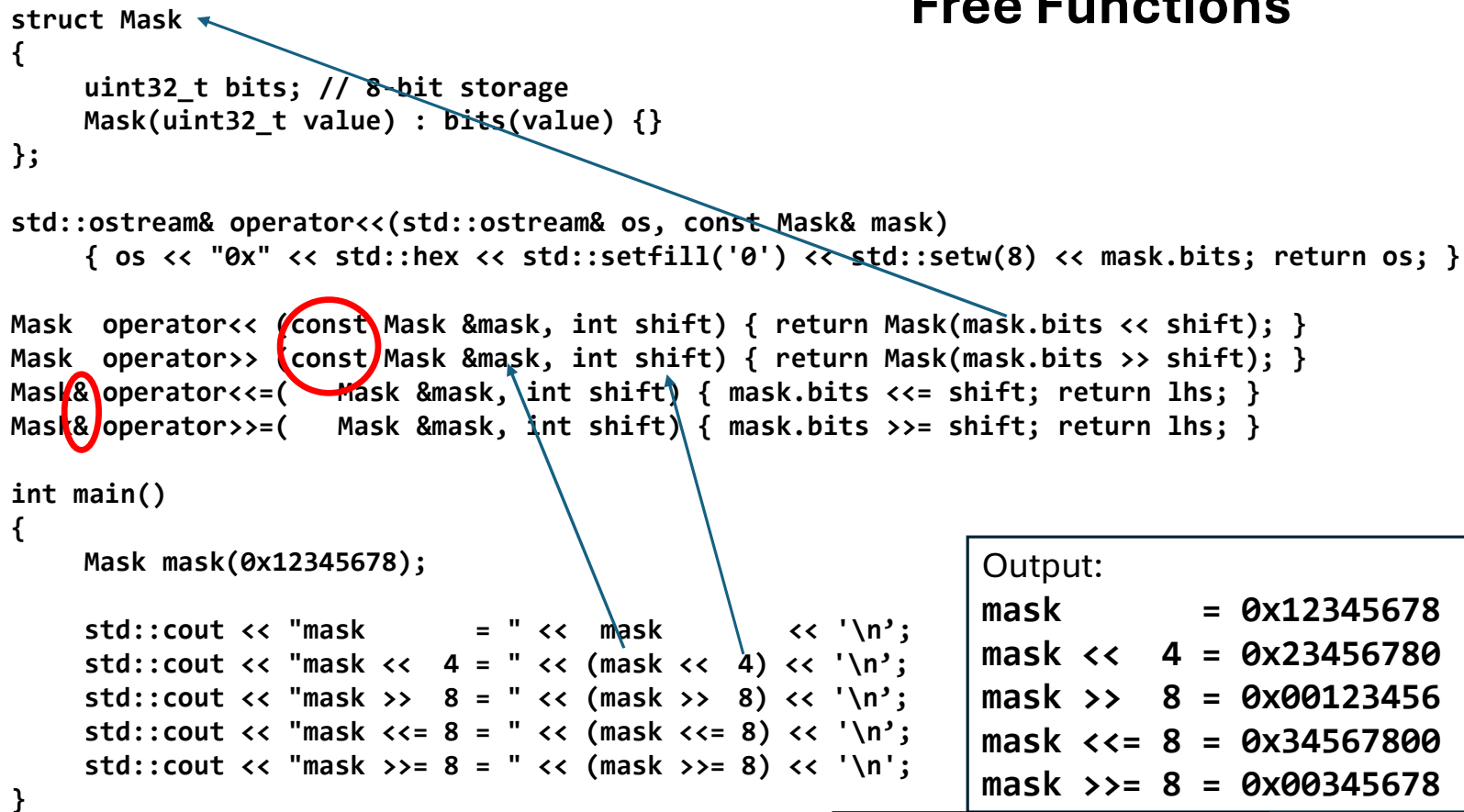
```
struct Mask
{
    uint32_t bits; // 8-bit storage
    Mask(uint32_t value) : bits(value) {}
};

std::ostream& operator<<(std::ostream& os, const Mask& mask)
{ os << "0x" << std::hex << std::setfill('0') << std::setw(8) << mask.bits; return os; }

Mask operator<<(const Mask &mask, int shift) { return Mask(mask.bits << shift); }
Mask operator>>(const Mask &mask, int shift) { return Mask(mask.bits >> shift); }
Mask& operator<<=(Mask &mask, int shift) { mask.bits <<= shift; return lhs; }
Mask& operator>>=(Mask &mask, int shift) { mask.bits >>= shift; return lhs; }

int main()
{
    Mask mask(0x12345678);

    std::cout << "mask      = " << mask << '\n';
    std::cout << "mask << 4 = " << (mask << 4) << '\n';
    std::cout << "mask >> 8 = " << (mask >> 8) << '\n';
    std::cout << "mask <<= 8 = " << (mask <<= 8) << '\n';
    std::cout << "mask >>= 8 = " << (mask >>= 8) << '\n';
}
```



Output:

```
mask      = 0x12345678
mask << 4 = 0x23456780
mask >> 8 = 0x00123456
mask <<= 8 = 0x34567800
mask >>= 8 = 0x00345678
```

godbolt.org/z/W383zzvve

B2B: Operator Overloading – <<, >> overloading

Member Functions

```
struct Mask
{
    uint32_t bits; // 8-bit storage
    Mask(uint32_t value) : bits(value) {}
    friend std::ostream& operator<<(std::ostream& os, const Mask& mask)
    { os << "0x" << std::hex << std::setfill('0') << std::setw(8) << mask.bits; return os; }

    Mask operator<< (int shift) const { return Mask(bits << shift); }
    Mask operator>> (int shift) const { return Mask(bits >> shift); }
    Mask& operator<<=(int shift) { bits <<= shift; return *this; }
    Mask& operator>>=(int shift) { bits >>= shift; return *this; }
};
```

```
int main()
{
    Mask mask(0x12345678);

    std::cout << "mask          = " << mask << '\n';
    std::cout << "mask << 4 = " << (mask << 4) << '\n';
    std::cout << "mask >> 8 = " << (mask >> 8) << '\n';
    std::cout << "mask <<= 8 = " << (mask <<= 8) << '\n';
    std::cout << "mask >>= 8 = " << (mask >>= 8) << '\n';
}
```

Output:

```
mask          = 0x12345678
mask << 4 = 0x23456780
mask >> 8 = 0x00123456
mask <<= 8 = 0x34567800
Mask >>= 8 = 0x00345678
```

godbolt.org/z/W41so59zY

B2B: Operator Overloading – <<, >> overloading

Hidden Friends (hybrid/D.R.Y. style)

```
struct Mask
{
    uint32_t bits;
    Mask(uint32_t value) : bits(value) {}
    friend std::ostream& operator<<(std::ostream& os, const Mask& mask)
        { os << "0x" << std::hex << mask.bits; return os; }

    Mask& LShift (int shift)          { bits << shift; return *this; }
    Mask& RShift (int shift)          { bits <> shift; return *this; }

    friend Mask operator<< (const Mask& mask, int shift) { Mask temp(mask); return temp.LShift(shift); }
    friend Mask operator>> (const Mask& mask, int shift) { Mask temp(mask); return temp.RShift(shift); }
    friend Mask& operator<<=(Mask& mask, int shift) { return mask.LShift(shift); }
    friend Mask& operator>>=(Mask& mask, int shift) { return mask.RShift(shift); }
};

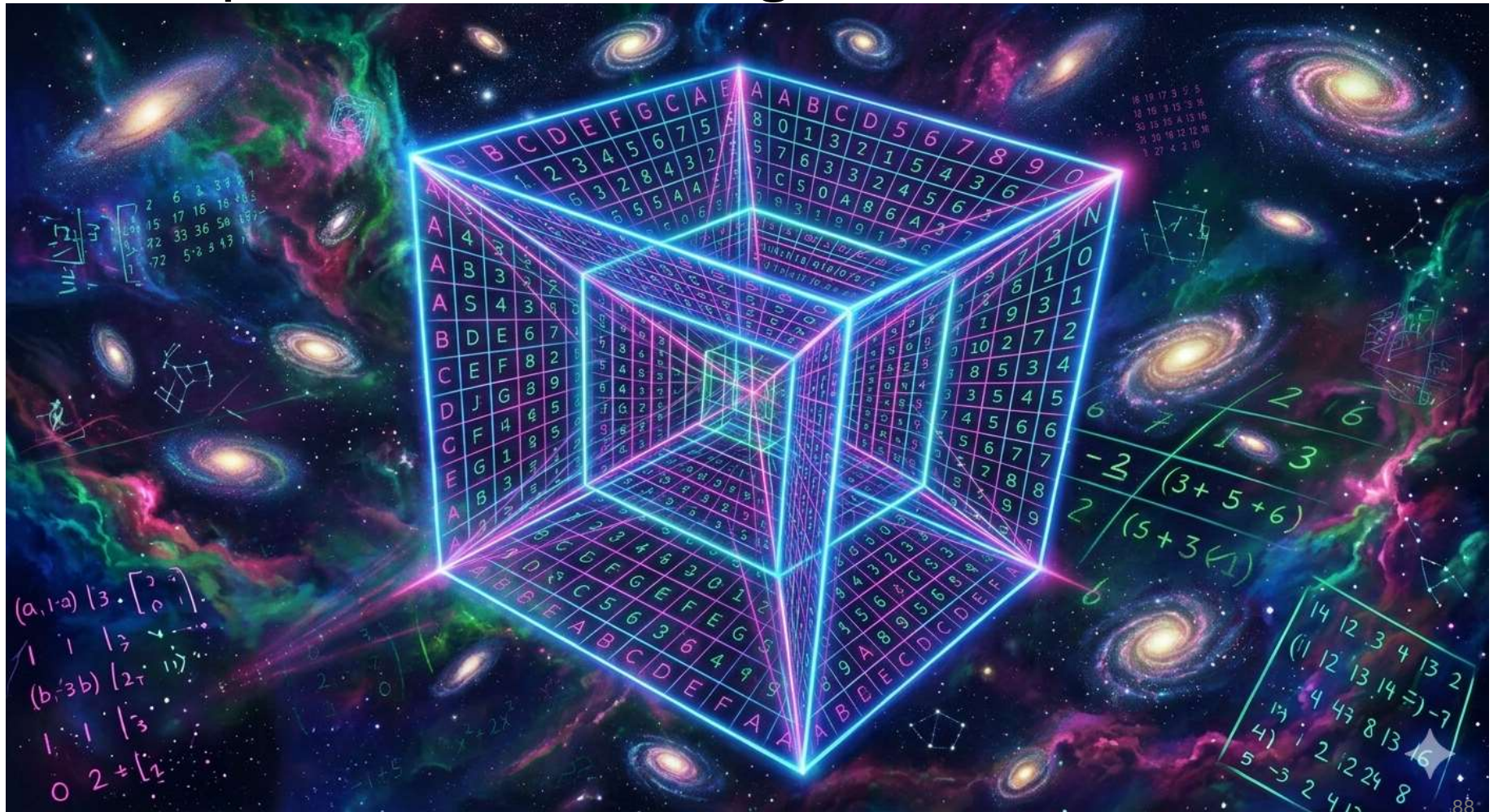
int main()
{
    Mask mask(0x12345678);
    std::cout << "mask          = " << mask          << '\n';
    std::cout << "mask << 4 = " << (mask << 4) << '\n';
    std::cout << "mask >> 8 = " << (mask >> 8) << '\n';
    std::cout << "mask <<= 8 = " << (mask <<= 8) << '\n';
    std::cout << "mask >>= 8 = " << (mask >>= 8) << '\n';
}
```

Output:

```
mask          = 0x12345678
mask << 4 = 0x23456780
mask >> 8 = 0x00123456
mask <<= 8 = 0x34567800
mask >>= 8 = 0x00345678
```

godbolt.org/z/MT3bErsdf

B2B: Operator Overloading – multidimensional indices



B2B: Operator Overloading – overloading operator[] (<C++23)

```
struct IntList
{
    std::vector<int> data;
    IntList() : data{10, 20, 30, 40, 50} {}

    // Must be a non-static member function with exactly ONE parameter
    int& operator[](size_t index)          { return data.at(index); }
    const int& operator[](size_t index) const { return data.at(index); }
};

//ERROR: friend & global free operator[] not allowed.
//int operator[](const IntList& obj, const int size_t) { return obj.data[index]; }
```

```
int main() {
    IntList list;

    std::cout << "list[2]: " << list[2] << '\n';
    list[2] = 99;
    std::cout << "list[2]: " << list[2] << '\n';
}
```

Output:
list[2]: 30
list[2]: 99

godbolt.org/z/zobefY5bY

B2B: Operator Overloading – overloading operator[,.] (C++23)

```
struct Array
{
    static const size_t X=4,Y=3,Z=2;
    int array[X*Y*Z] = {111,211,311,411, 121,221,321,421, 131,231,331,431,
                        112,212,312,412, 122,222,322,422, 132,232,332,432};
    size_t CalcIndex(size_t x, size_t y, size_t z) const { return z*X*Y + y*X + x; }

    int& operator[](size_t x, size_t y, size_t z) { return array[CalcIndex(x,y,z)]; }
    const int& operator[](size_t x, size_t y, size_t z) const { return array[CalcIndex(x,y,z)]; }
};

int main()
{
    Array array;
    std::cout << "xyz xyz xyz xyz\n";
    for(size_t z{}; z < Array::Z; z++)
    {
        for(size_t y{}; y < Array::Y; y++)
        {
            for(size_t x{}; x < Array::X; x++)
                std::cout << array[x, y, z] << ' ';
            std::cout << '\n';
        }
        std::cout << '\n';
    }
}
```

Output:

```
xyz xyz xyz xyz
111 211 311 411
121 221 321 421
131 231 331 431

112 212 312 412
122 222 322 422
132 232 332 432
```

godbolt.org/z/zPf85nP5o

B2B: Operator Overloading – fake operator[,.] < C++23

```
#if (__cpp_multidimensional_subscript>=202110L) //feature test macro, if >=C++23
#define operatorMD operator[]
#define FAKE ""
#else
#define FAKE "FAKE "
#endif

template<typename T, std::size_t Z, std::size_t Y, std::size_t X>
struct Array3d
{
    std::array<T, X * Y * Z> m{};
    constexpr T& operatorMD(size_t z, size_t y, size_t x)
        { return m[z * Y * X + y * X + x]; }
};

int main()
{
    Array3d<int, 4, 3, 2> array;

    array.operatorMD(3, 2, 1) = 42;    // surrogate for array[3, 2, 1] = 42;
    std::cout << FAKE << "v[3, 2, 1] = " << array.operatorMD(3, 2, 1) << '\n';
}
```

Output: -std=c++23

v[3, 2, 1] = 42

Output: -std=c++14

FAKE v[3, 2, 1] = 42

godbolt.org/z/Exo4s39Yn

B2B: Operator Overloading – stream(ing) operators <<, >>



B2B: Operator Overloading –

```
struct Foo
{
    std::string string{};
    Foo(std::string string) : string(string) {}

    std::ostream& TextOut(std::ostream& os) const
        { return os << string; }

    // Global OR friend not both.
    // friend std::ostream& operator<<(std::ostream& os, const Foo& foo)
    //     { return foo.TextOut(os); }
};

std::ostream& operator<<(std::ostream& os, const Foo& foo)
    { return os << foo.TextOut(os); }

int main()
{
    Foo foo("this is a test");
    std::cout << foo;
}
```

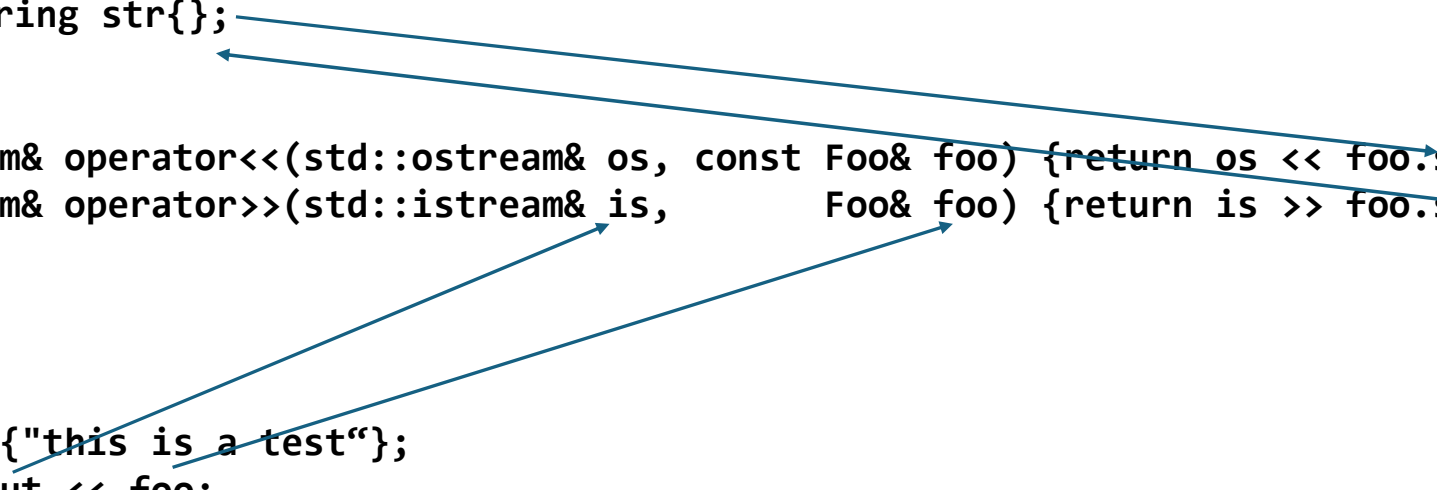
B2B: Operator Overloading –

Stream <<,>> operators Member Functions

```
struct Foo
{
    std::string str{};
};

std::ostream& operator<<(std::ostream& os, const Foo& foo) {return os << foo.str;}
std::istream& operator>>(std::istream& is,      Foo& foo) {return is >> foo.str;}

int main()
{
    Foo foo{"this is a test"};
    std::cout << foo;
    std::cin >> foo;
}
```



B2B: Operator Overloading –

```
struct Base
{
    std::string string{};
    Base(std::string string) : string(string) {}

    virtual std::ostream& TextOut(std::ostream& os) const { os << string; return os; }
    friend std::ostream& operator<<(std::ostream& os, const Base& base) {return base.TextOut(os);}
};
struct Derived : public Base
{
    int i;
    Derived(std::string string, int i) : Base(string), i(i) {}

    virtual std::ostream& TextOut(std::ostream& os) const { return Base::TextOut(os) << ' ' << i;}
};
int main()
{
    Base base("this is a base test");
    std::cout << base <<'\n';

    Derived derived("this is a derived test", 42);
    std::cout << derived <<'\n';
}
```

Output:

this is a base test
this is a derived test 42

godbolt.org/z/ndafasnGM

B2B: Operator Overloading –

Stream <<,>> operators Friend Functions

```
struct Foo
{
    std::string str{};

    friend std::ostream& operator<<(std::ostream& os, const Foo& foo){return os << foo.str;}
    friend std::istream& operator>>(std::istream& is, Foo& foo){return is >> foo.str;}
};

int main()
{
    Foo foo{"this is a test"};
    std::cout << foo;
    std::cin >> foo;
}
```

The diagram illustrates the resolution of stream operators. Four blue arrows originate from the `main` function and point to the corresponding operator functions in the `Foo` struct. The first arrow points from `std::cout << foo;` to `operator<<(std::ostream& os, const Foo& foo)`. The second arrow points from `std::cin >> foo;` to `operator>>(std::istream& is, Foo& foo)`. The third arrow points from the `str` member access in `foo.str` within the `operator<<` function back to the `std::string str` member in the `Foo` struct. The fourth arrow points from the `str` member access in `foo.str` within the `operator>>` function back to the `std::string str` member in the `Foo` struct.

B2B: Operator Overloading – tree example friend operator<<

```
template<typename Type>
class Node
{
    Type    data{};
    std::shared_ptr<Node> left, right;

    friend std::ostream& operator<<(std::ostream& os, const std::shared_ptr<Node>& node)
    {
        if (node->left) os << node->left;
        os << node->data << ", ";
        if (node->right) os << node->right;
        return os;
    }
public:
    template<typename T> Node(T rlm, T lmr, int count) {...}
};

template<typename type, int count, int count2>
auto RebuildTree(const type(&rlm)[count], const type(&lmr)[count2]) {...}

int main()
{
    int lmr[] = {3,5,6,7,10,12,13,15,16,18,20,23};
    int lrm[] = {3,7,6,10,13,12,5,18,23,20,16,15};
    std::cout << RebuildTree(lrm, lmr) << "\n";
}
```

Output:

3, 5, 6, 7, 10, 12, 13, 15, 16, 18, 20, 23,

godbolt.org/z/3rsrY6K66

B2B: Function Overloading – tree example member operator<<

```
template<typename Type>
class Node
{
    Type    data;
    std::shared_ptr<Node> left, right;
public:
    std::ostream& operator<<(std::ostream& os) const
    {
        if (left)  *left  << os;
        os << data << ", ";
        if (right) *right << os;
        return os;
    }
    template<typename T> Node(T rlm, T lrm, int count) {...}
};
template<typename type, int count, int count2>
auto RebuildTree(const type(&rlm)[count], const type(&lrm)[count2]) {...}
int main()
{
    int mlr[] = {3,5,6,7,10,12,13,15,16,18,20,23};
    int lmr[] = {3,7,6,10,13,12,5,18,23,20,16,15};
    auto tree = RebuildTree(lrm, lmr);
    *tree << std::cout;
    std::cout << "\n";
}
```

Bad & Ugly
Don't do this !!

Output:

3, 5, 6, 7, 10, 12, 13, 15, 16, 18, 20, 23,

godbolt.org/z/W4fWGfqP6

B2B: Operator Overloading –

Stream <<,>> operators Member Functions

```
struct Foo
{
    std::string str{};

    std::ostream& operator<<(std::ostream& os) {return os << str;}
    std::istream& operator>>(std::istream& is) {return is >> str;}
};

int main()
{
    Foo foo{"this is a test"};
    std::cout << foo;
    std::cin >> foo;
    foo << std::cout;
    foo >> std::cin;
}
```

B2B: Operator Overloading – tree example friend operator<<

```
template<typename Type>
class Node
{
    Type    data;
    std::shared_ptr<Node> left, right;

    std::ostream& TextOut(std::ostream& os) const
    {
        if (left) os << left;
        os << data << ", ";
        if (right) os << right;
        return os;
    }
    friend std::ostream& operator<<(std::ostream& os, const std::shared_ptr<Node>& node)
        { return node->TextOut(os); }
public:
    template<typename T> Node(T rlm, T lmr, int count) {...}
};
template<typename type, int count, int count2>
auto RebuildTree(const type(&rlm)[count], const type(&lmr)[count2]) {...}
int main()
{
    int lmr[] = {3,5,6,7,10,12,13,15,16,18,20,23};
    int lrm[] = {3,7,6,10,13,12,5,18,23,20,16,15};
    std::cout << RebuildTree(lrm, lmr) << "\n";
}
```

Output:

3, 5, 6, 7, 10, 12, 13, 15, 16, 18, 20, 23,

godbolt.org/z/Tfz7aK6Gn

B2B: Operator Overloading – in the out door

```
#include <iostream>
#include <string>

int main()
{
    int age;
    double height;
    std::string name;

    std::cout << "Enter your first name, age, and height (e.g., Alex 25 5.9): ";

    std::cin >> name >> age >> height;

    std::cout << "\nHello " << name << ". You are " << age
        << " years old and " << height << " feet tall.\n";
}
```

Output: Enter your first name, age, and height (e.g., Alex 25 5.9):

Input: Bob 25 6.1

Output: Hello Bob. You are 25 years old and 6.1 feet tall.

godbolt.org/z/Tfz7aK6Gn

B2B: Operator Overloading – alien archive



B2B: Operator Overloading – off brand streams

```
int main()
{
    {
        std::cout << "saving\n";
        Archive arc("filename.arc", Mode::save);

        Bar bar1("bob", 1.2, 42);
        std::cout << bar1 << '\n';
        arc << bar1;
    }
    {
        std::cout << "loading\n";
        Archive arc("filename.arc", Mode::load);

        Bar bar2;
        arc >> bar2;
        std::cout << bar2 << '\n';
    }
}
```

Output:
saving
bob 1.2 42
loading
bob 1.2 42

godbolt.org/z/e57oTT7j8

B2B: Operator Overloading – serializable class

```
struct Bar
{
    Foo foo{};
    int i{};

    Bar() {}
    Bar(const char* name, double d, int i) :foo(name, d), i(i) {}

    Archive& Serialize(Archive& arc)
    {
        if (arc.IsLoading())
            arc >> foo >> i;
        else
            arc << foo << i;
        return arc;
    }
    friend std::ostream& operator<<(std::ostream& os, Bar& bar)
        { return os << bar.foo << ' ' << bar.i; }
};
```

godbolt.org/z/e57oTT7j8

B2B: Operator Overloading – another serializable class

```
struct Foo
{
    char name[20]{};
    double d{};

    Foo() {}
    Foo(const char* str, double d) :d(d) { std::strncpy(name,str,sizeof(name)); }

    Archive& Serialize(Archive& arc) const
    {
        if (arc.IsLoading())
            arc >> name >> d;
        else
            arc << name << d;
        return arc;
    }
    friend std::ostream& operator<<(std::ostream& os, Foo& foo)
        { return os << foo.name << ' ' << foo.d; }
};
```

godbolt.org/z/e57oTT7j8

B2B: Operator Overloading – magic behind the scenes

```
struct Archive;
template <typename T>
concept Serializable = requires(T t, Archive arc) { t.Serialize(arc); };
enum Mode { load = std::ios::in, save = std::ios::out };

struct Archive
{
    Mode mode{load};
    std::fstream file{};

    Archive() {}
    Archive(std::string name, Mode mode) :mode(mode),
        file(name, std::ios_base::openmode(mode) | std::ios::binary) {}
    ~Archive() { file.close(); }
    bool IsLoading() { return mode == load; }

    friend Archive& operator<<(Archive& arc, Serializable auto& t) { return t.Serialize(arc); }
    friend Archive& operator>>(Archive& arc, Serializable auto& t) { return t.Serialize(arc); }

    template<typename T>
    friend Archive& operator<<(Archive& arc, const T& t) requires (!Serializable<T>)
        { arc.file.write((const char*)&t, sizeof(t)); return arc; }

    template<typename T>
    friend Archive& operator>>(Archive& arc, const T& t) requires (!Serializable<T>)
        { arc.file.read((char*)&t, sizeof(t)); return arc; }
};
```

godbolt.org/z/e57oTT7j8

B2B: Operator Overloading – 3D - matrix math



B2B: Operator Overloading – 3D - data types and operators

Types:

Matrix (4x4)
Vector (1x4) (xyz,w=0)
Point (1x4) (xyz,w=1)

Matrix operators

```
matrix = matrix * matrix;  
matrix *= matrix;
```

Vector operators

```
vector = vector * matrix;  
vector *= matrix;  
vector = vector * vector; // cross product  
vector *= vector;        // cross product  
vector = Normalize(vector);
```

Point operators

```
point = point * matrix;  
point *= matrix;  
point = point + vector;  
point vector;  
vector = point - point;  
point.PerspectiveDivide(); // 3D→2D
```

Manipulator matrices

```
Matrix Identity();  
Matrix RotateX(float degrees);  
Matrix RotateY(float degrees);  
Matrix RotateZ(float degrees);  
Matrix Scale(float x, float y, float z);  
Matrix Translate(float x, float y, float z);
```

Matrix transforms // 3D→2D

```
Matrix PointOfView(const Point& eye, const Point& target,  
                  const Vector& up);
```

```
Matrix FieldOfView(float fovAngle, float aspectRatio,  
                  float nearPlaneDistance, float farPlaneDistance);
```

```
Matrix Viewport(Rect& view, float minZ, float maxZ);
```

Matrix, Vector, Point: All type correct and type coherent

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D - example - matrix

```
class Matrix
{
    union {struct { float mat[4][4]; };
          struct { float mat[16]; } data{}};
    Matrix Multiply(const Matrix& rhs) const {...}
public:
    Matrix() {};
    ~Matrix() {};

    Matrix(const Matrix& rhs) : data(rhs.data) {}
    Matrix(std::initializer_list<float[4]> list);

    Matrix&operator=(const Matrix& rhs) { data = rhs.data; return *this; }
    float& operator[](uint r, uint c) { return mat[r][c]; }
    const float& operator[](uint r, uint c) const { return mat[r][c]; }

    friend Matrix operator* (const Matrix& lhs, const Matrix& rhs)
                                { return lhs.Multiply(rhs); }
    friend Matrix& operator*=(      Matrix& lhs, const Matrix& rhs)
                                { lhs = lhs.Multiply(rhs); return lhs;}
};
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D – manipulator and transforms

```
class Matrix
{
public:
    Matrix(std::initializer_list<float[4]> list) {...}
;

Matrix RotateX    (float degrees) {...}
Matrix RotateY    (float degrees) {...}
Matrix RotateZ    (float degrees)
{
    float c = cos  (degrees * pi / 180);
    float s = sin  (degrees * pi / 180);
    return { { c, s, 0, 0 },
              { -s, c, 0, 0 },
              { 0, 0, 1, 0 },
              { 0, 0, 0, 1 } };
}
Matrix Scale      (float x, float y, float z) {...}
Matrix Translate(float x, float y, float z) {...}

Matrix PointOfView(const Point& eye, const Point& target, const Vector& up) {...}
Matrix FieldOfView(float fovAngle, float aspectRatio, float nearPlane, float farPlane) {...}
Matrix Viewport   (Rect& view, float minZ, float maxZ) {...}
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D – vector/point base data

```
namespace
{
    class Data4
    {
    protected:
        float x{}, y{}, z{}, w{};
        void Multiply(const Matrix& rhs);
    public:
        Data4() {}
        Data4(const Data4& rhs) : data(rhs.data) {}
        Data4(float x, float y, float z, float w = 0) : x(x), y(y), z(z), w(w) {}
        Data4& operator = (const Data4& rhs) { data = rhs.data; return *this; }
    };
}
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D– class vector : data

```
Namespace{
    class Data4 {
    protected:
        void Multiply(const Matrix& rhs) {...}
    public:
        Data4& operator = (const Data4& rhs) {...}
    };
}
class Vector : public Data4
{
    Vector CrossProduct(const Vector& rhs) const {...}
    Vector Normalize() const {...}
public:
    Vector() {}
    Vector(const Vector& rhs) : Data4(rhs) {}
    Vector(float x, float y, float z) : Data4(x, y, z, 0) {}

    friend Vector  Normalize (const Vector& rhs
                               ) { return rhs.Normalize(); }
    friend Vector  operator * (const Vector& lhs, const Matrix& rhs) { Vector ret = lhs; ret.Multiply(rhs); return ret; }
    friend Vector& operator*= (    Vector& lhs, const Matrix& rhs) {          lhs.Multiply(rhs); return lhs; }
    friend Vector  operator * (const Vector& lhs, const Vector& rhs) { return lhs.CrossProduct(rhs); }
    friend Vector& operator*= (    Vector& lhs, const Vector& rhs) {  lhs = lhs.CrossProduct(rhs);    return lhs; }
};
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D– class point : data

```
Namespace{
    class Data4{
    protected:
        void Multiply(const Matrix& rhs) {...}
    public:
        Data4& operator = (const Data4& rhs) {...}
    };
}
class Point : public Data4
{
    Point Add      (const Vector& rhs) const {...}
    Vector Subtract(const Point&  rhs) const {...}
public:
    Point() {}
    Point(const Point& rhs)    : Data4(rhs) {}
    Point(float x, float y, float z) : Data4(x, y, z, 1) {}

    void PerspectiveDivide() {...}
    friend Point  operator * (const Point& lhs, const Matrix& rhs) { Point ret = lhs; ret.Multiply(rhs); return ret; }
    friend Point& operator*=(      Point& lhs, const Matrix& rhs) {          lhs.Multiply(rhs); return lhs; }
    friend Point  operator + (const Point& lhs, const Vector& rhs) { return lhs.Add(rhs); }
    friend Point& operator  (      Point& lhs, const Vector& rhs) { lhs =  lhs.Add(rhs); return lhs; }
    friend Vector operator - (const Point& lhs, const Point&  rhs) { return lhs.Subtract(rhs); }
};
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – 3D– matrix math

```
int main()
{
    using namespace D3;
    auto print=[](const char* str, Point &point)
        {std::print("{} point={:3},{:3},{:3}\n", str, (int)point.X(), (int)point.Y(), (int)point.Z());};

    Point point{100,50,25};
    print("start", point);

    point*=RotateZ(90);
    print("RotateZ(90) around Z axis", point);

    point*=RotateY(90);
    print("RotateY(90) around Y axis", point);

    point*=RotateX(90);
    print("RotateX(90) around X axis", point);

    point*=Scale(1.0, 0.5, 0.5);
    print("Scale(1.0, 0.5, 0.5)", point);

    point*=Translate(50, 25, 100);
    print("Translate(50, 25, 100)", point);
}
```

Output:

```
start point=[100, 50, 25]
RotateZ(90) around Z axis point=[-50, 100, 25]
RotateY(90) around Y axis point=[ 25, 100, 50]
RotateX(90) around X axis point=[ 25, -50, 100]
Scale(1.0, 0.5, 0.5) point=[ 25, -25, 50]
Translate(50, 25, 100) point=[ 75, 0, 150]
```

godbolt.org/z/8hnPsoMTh

B2B: Operator Overloading – streams matrix example



B2B: Operator Overloading – stream operator<<

```
class Matrix
{
    virtual std::ostream& operator<<((std::ostream& os) const
    {
        os << std::format("Matrix: [ [ {:2},{:2},{:2},{:2} ]\n", int(mat[0][0]), int(mat[0][1]), int(mat[0][2]), int(mat[0][3]));
        os << std::format("      [ {:2},{:2},{:2},{:2} ]\n", int(mat[1][0]), int(mat[1][1]), int(mat[1][2]), int(mat[1][3]));
        os << std::format("      [ {:2},{:2},{:2},{:2} ]\n", int(mat[2][0]), int(mat[2][1]), int(mat[2][2]), int(mat[2][3]));
        os << std::format("      [ {:2},{:2},{:2},{:2} ]]", int(mat[3][0]), int(mat[3][1]), int(mat[3][2]), int(mat[3][3]));
        return os;
    }
    friend std::ostream& operator<<(std::ostream& os, const Matrix& matrix) //os << data;
    { return matrix.operator<<(os); }
};
```

```
std::cout << matrix;
```

```
Matrix: [ [ 0, 1, 0, 0 ]
          [ -1, 0, 0, 0 ]
          [ 0, 0, 1, 0 ]
          [ 0, 0, 0, 1 ] ]
```

godbolt.org/z/s7q3bqMMn

B2B: Operator Overloading – stream operator<<

```
class Data4
{
    virtual std::ostream& operator<<(std::ostream& os) const
        {return os << std::format("[{:3},{:3},{:3},{:3}]",int(x),int(y),int(z),int(w)); }

    friend std::ostream& operator<<(std::ostream& os, const Data4& data)
        { return data.operator<<(os); }
};

class Vector : public Data4
{
    virtual std::ostream& operator<<(std::ostream& os) const
        { os << "Vector: "; return Data4::operator<<(os); }
};

std::cout << vector;

Vector: [ 50, 25,100,  0]
```

godbolt.org/z/s7q3bqMMn

B2B: Operator Overloading – stream operator<<

```
class Data4
{
    virtual std::ostream& operator<<(std::ostream& os) const
        {return os << std::format("[{:3},{:3},{:3},{:3}]",int(x),int(y),int(z),int(w)); }

    friend std::ostream& operator<<(std::ostream& os, const Data4& data)
        { return data.operator<<(os); }
};

class Point : public Data4
{
    virtual std::ostream& operator<<(std::ostream& os) const
        { os << "Point: "; return Data4::operator<<(os); }
};

std::cout << point;

Point: [ 25,100, 50,  1]
```

godbolt.org/z/s7q3bqMMn

B2B: Operator Overloading – stream operator<<

```
int main()
{
    D3::Point point{100,50,25};
    std::cout << "Point point{100,50,25}\n";
    std::cout << point << "\n\n";

    D3::Vector vector{50, 25, 100};
    std::cout << "Vector vector{50, 25, 100}\n ";
    std::cout << vector << "\n\n";

    D3::Matrix rotate = D3::RotateZ(90);
    std::cout << " Matrix rotate = RotateZ(90)\n ";
    std::cout << rotate << "\n\n";

    point vector;
    std::cout << "point vector;\n ";
    std::cout << point << "\n\n";

    point *= rotate;
    std::cout << "point *= rotate //around Z axis\n ";
    std::cout << point << "\n\n";
}
```

Output:

Point point{100,50,25}
Point: [100, 50, 25, 1]

Vector vector{50, 25, 100}
Vector: [50, 25,100, 0]

Matrix rotate = RotateZ(90)
Matrix: [[0, 1, 0, 0]
 [-1, 0, 0, 0]
 [0, 0, 1, 0]
 [0, 0, 0, 1]]

point vector;
Point: [150, 75,125, 1]

point *= rotate //around Z axis
Point: [-75,150,125, 1]

godbolt.org/z/s7q3bqMMn

B2B: Operator Overloading –relations and flying saucers



B2B: Operator Overloading – maintaining good relations

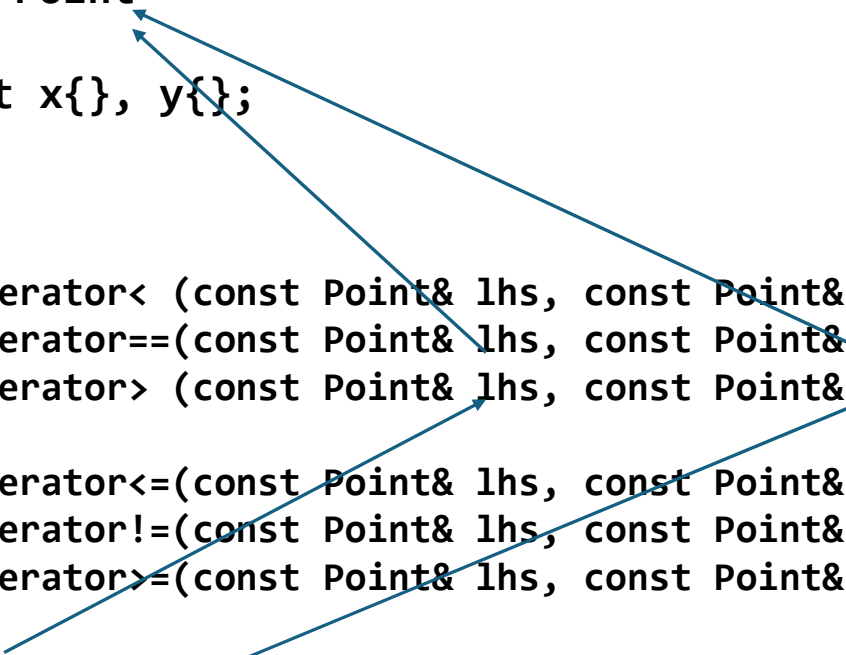
Relational Operators: <, >, <=, >=, ==, != – Free functions:

```
struct Point
{
    int x{}, y{};
};

bool operator< (const Point& lhs, const Point& rhs) {...} // less than
bool operator==(const Point& lhs, const Point& rhs) {...} // equal
bool operator> (const Point& lhs, const Point& rhs) {...} // greater than

bool operator<=(const Point& lhs, const Point& rhs) {...} // less than or equal
bool operator!=(const Point& lhs, const Point& rhs) {...} // not equal
bool operator>=(const Point& lhs, const Point& rhs) {...} // greater than or equal

if(Point{2,3} > Point{3,2}) {...} else {...} // same as operator>(Point{2,3},Point{3,2})
```

A diagram consisting of four blue arrows. One arrow originates from the 'Point' struct definition and points to the 'lhs' parameter of the 'operator<' function. Another arrow originates from the 'Point' struct and points to the 'rhs' parameter of the 'operator<' function. A third arrow originates from the 'Point' struct and points to the 'lhs' parameter of the 'operator>' function. A fourth arrow originates from the 'Point' struct and points to the 'rhs' parameter of the 'operator>' function.

B2B: Operator Overloading – basics of good relations

Relational Operators: <, >, <=, >=, ==, != – Free functions:

```
struct Point
{
    int x{}, y{};
};
```

```
Bool operator< (const Point& lhs, const Point& rhs) {return (lhs.x< rhs.x) || (lhs.y< rhs.y); }
Bool operator==(const Point& lhs, const Point& rhs) {return (lhs.x==rhs.x) && (lhs.y==rhs.y); }
```

```
bool operator> (const Point& lhs, const Point& rhs) {...} // greater than
bool operator<=(const Point& lhs, const Point& rhs) {...} // less than or equal
bool operator!=(const Point& lhs, const Point& rhs) {...} // not equal
bool operator>=(const Point& lhs, const Point& rhs) {...} // greater than or equal
```

B2B: Operator Overloading – good relations get complicated

Relational Operators: <, >, <=, >=, ==, != – Free functions:

```
struct Point
{
    int x{}, y{};
};
```

```
bool operator< (const Point& lhs, const Point& rhs) {return (lhs.x <  rhs.x) || (lhs.y <  rhs.y); }
bool operator==(const Point& lhs, const Point& rhs) {return (lhs.x == rhs.x) && (lhs.y == rhs.y); }
```

```
bool operator> (const Point& lhs, const Point& rhs) {return !operator< (lhs, rhs) && !operator==(lhs, rhs); }
bool operator<=(const Point& lhs, const Point& rhs) {return  operator< (lhs, rhs) ||  operator==(lhs, rhs); }
bool operator!=(const Point& lhs, const Point& rhs) {return                                     !operator==(lhs, rhs); }
bool operator>=(const Point& lhs, const Point& rhs) {return !operator< (lhs, rhs);                                     }
```

B2B: Operator Overloading – aliens to the rescue

Relational Operators: <, ==, >, <=, !=, >=, <=> – Free functions:

```
struct Point
{
    int x{}, y{};
};

Bool operator< (const Point& lhs, const Point& rhs) {return (lhs.x< rhs.x) || (lhs.y< rhs.y); }
Bool operator==(const Point& lhs, const Point& rhs) {return (lhs.x==rhs.x) && (lhs.y==rhs.y); }

// <=> auto generates <, ==, >, <=, !=, >=
auto operator<=>(const Point& lhs, const Point& rhs)
{
    if(operator< (lhs, rhs)) return std::strong_ordering::less;
    if(operator==(lhs, rhs)) return std::strong_ordering::equivalent;
    return std::strong_ordering::greater;
}

if(Point{2,3} > Point{3,2}) {...} else {...} // same as operator<=>(Point{2,3},Point{3,2})
```

B2B: Operator Overloading – three-way comparison operator

The Spaceship Operator: <, ==, <=> – Free functions:

Type: `std::strong_ordering` // Example string comparison case sensitive

```
auto operator<=> (const T& lhs, const T& rhs)
```

```
{
```

```
    if( operator < (lhs, rhs)) return std::strong_ordering::less;
```

```
    if( operator == (lhs, rhs)) return std::strong_ordering::equivalent;
```

```
        return std::strong_ordering::greater;
```

```
}
```

Type: `std::partial_ordering` // Example string comparison ignoring diacritics

```
< 0      std::partial_ordering::less
```

// or a point struct, comparing x and ignoring y

```
== 0     std::partial_ordering::equivalent
```

```
> 0      std::partial_ordering::greater
```

Type: `std::weak_ordering` // Example string comparison case insensitive

```
< 0      std::weak_ordering::less
```

```
== 0     std::weak_ordering::equivalent
```

```
> 0      std::weak_ordering::greater
```

Relational Operators: <, ==, >, <=, !=, >=, <=> – Member functions:

```
struct Point
{
    int x{}, y{};
    bool operator< (const Point& rhs) { return (x< rhs.x) || (y< rhs.y); }
    bool operator==(const Point& rhs) { return (x==rhs.x) && (y==rhs.y); }

    // <=> auto generates <, ==, >, <=, !=, >=
    auto operator<=>(const Point& rhs) = default;
    // or
    auto operator<=>(const Point& rhs)
    {
        if(operator< (rhs)) return std::strong_ordering::less;
        if(operator==(rhs)) return std::strong_ordering::equivalent;
        return std::strong_ordering::greater;
    }
};

if(Point{2,3} > Point{3,2}) {...} else {...} // same as Point{2,3}.operator<=>(Point{3,2})
```

B2B: Operator Overloading – testing <, ==, >, <=, !=, >=, <=>

```
struct Point {
    int x{}, y{};

    friend bool operator< (const Point& lhs, const Point& rhs) { return lhs.LT(rhs); }
    friend bool operator==(const Point& lhs, const Point& rhs) { return lhs.EQ(rhs); }

    friend auto operator<=>(const Point& lhs, const Point& rhs)
    { // <=> auto generates <, ==, >, <=, !=, >=
        if(lhs.LT(rhs)) return std::strong_ordering::less;
        if(lhs.EQ(rhs)) return std::strong_ordering::equivalent;
        return std::strong_ordering::greater;
    }
    bool LT(const Point& pt) const { return (x< pt.x) || (y< pt.y); }
    bool EQ(const Point& pt) const { return (x==pt.x) && (y==pt.y); }
    bool GT(const Point& pt) const { return !LT(pt) && !EQ(pt); }
    bool LE(const Point& pt) const { return LT(pt) || EQ(pt); }
    bool NE(const Point& pt) const { return !EQ(pt); }
    bool GE(const Point& pt) const { return !LT(pt); }
};

If(Point{2,3} > Point{3,2}) {...} else {...} // same as operator<(Point{2,3},Point{3,2})
```

godbolt.org/z/dMz1jro7r

B2B: Operator Overloading – testing <, ==, >, <=, !=, >=, <=>

```
void Test(const Point& p1, const Point& p2)
```

```
{
    std::cout << std::format("p1({},{}), p2({},{})\n", p1.x, p1.y, p2.x, p2.y);
    std::cout << std::format("p1 < p2 {:5}\t\t p1.LT (p2) {:5}\t\t (p1 <=> p2) < 0 {:5}\n", p1 < p2, p1.LT(p2), p1 <=> p2 < 0);
    std::cout << std::format("p1 == p2 {:5}\t\t p1.EQ(p2) {:5}\t\t (p1 <=> p2) == 0 {:5}\n", p1 == p2, p1.EQ(p2), p1 <=> p2 == 0);
    std::cout << std::format("p1 > p2 {:5}\t\t p1.GT (p2) {:5}\t\t (p1 <=> p2) > 0 {:5}\n", p1 > p2, p1.GT(p2), p1 <=> p2 > 0);
    std::cout << std::format("p1 <= p2 {:5}\t\t p1.LE (p2) {:5}\t\t (p1 <=> p2) <= 0 {:5}\n", p1 <= p2, p1.LE(p2), p1 <=> p2 <= 0);
    std::cout << std::format("p1 != p2 {:5}\t\t p1.NE(p2) {:5}\t\t (p1 <=> p2) != 0 {:5}\n", p1 != p2, p1.NE(p2), p1 <=> p2 != 0);
    std::cout << std::format("p1 >= p2 {:5}\t\t p1.GE(p2) {:5}\t\t (p1 <=> p2) >= 0 {:5}\n", p1 >= p2, p1.GE(p2), p1 <=> p2 >= 0);
    std::cout << '\n';
}
int main()
{
    Test({2,2}, {3,3});
    Test({3,2}, {2,3});
    Test({3,3}, {3,3});
    Test({3,4}, {4,3});
    Test({3,3}, {3,2});
}
```

Output:

p1(3,2), p2(2,3)

p1 < p2 true	p1.LT (p2)true	(p1 <=> p2) < 0 true
p1 == p2 false	p1.EQ (p2>false	(p1 <=> p2) == 0 false
p1 > p2 false	p1.GT (p2>false	(p1 <=> p2) > 0 false
p1 <= p2 true	p1.LE (p2>true	(p1 <=> p2) <= 0 true
p1 != p2 true	p1.NE (p2>true	(p1 <=> p2) != 0 true
p1 >= p2 false	p1.GE (p2>false	(p1 <=> p2) >= 0 false

p1(3,3), p2(3,3)

p1 < p2 false	p1.LT (p2>false	(p1 <=> p2) < 0 false
p1 == p2 true	p1.EQ (p2>true	(p1 <=> p2) == 0 true ...more

godbolt.org/z/dMz1jro7r

B2B: Operator Overloading – chroma-cast cask



B2B: Operator Overloading – example cast operator 1 of 4

```
class CustomString
{
    std::string data;
public:
    CustomString(std::string str) : data(std::move(str)) {}

    // Overloaded cast operator to raw C-string
    operator const char*() const { return data.c_str(); }
};

void LegacyCFunction(const char* str) { /* ... */ }

int main() {
    CustomString myStr("Hello World");
    LegacyCFunction(myStr); // Implicitly converted to const char*
}
```

B2B: Operator Overloading – example cast operator 2 of 4

```
template <typename T>
class SmartRef
{
private:
    T& ref;
public:
    SmartRef(T& ref) : ref(ref) {}

    // Allows the smart reference to be used anywhere a raw pointer is expected
    operator T*() const { return &ref; }
};

void LegacyCFunction(const int* pint) { /* ... */ }

Int main()
{
    int i{};
    SmartRef sri(i);
    LegacyCFunction(sri);
}
```

B2B: Operator Overloading – example cast operator 3 of 4

```
class DatabaseConnection
{
    bool is_connected{};
public:
    // "explicit" prevents accidental implicit conversions to integers
    explicit operator bool() const { return is_connected; }
    bool IsConnect()          const { return is_connected; }
};

int main()
{
    DatabaseConnection db;
    if(db) // cleaner to do if(db.IsConnect())
    { /* Safe: Evaluates contextually in the 'if' condition */
        // int x = db; // Error! Prevented by the "explicit" keyword
    }
}
```

B2B: Operator Overloading – example cast operator 4 of 4

```
class Fraction
{
    int numerator;
    int denominator;
public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}
    int N() { return numerator; }
    int D() { return denominator; }

    // Convert to double for floating-point calculations
    explicit operator double() const { return double(numerator) / denominator; }
    double GetDecimal() const { return double(numerator) / denominator; }
};

int main()
{
    Fraction frac(3,4);
    std::print("{} = {} / {} \n" , frac.GetDecimal(), frac.N(), frac.D());
    std::print("{} = {} / {} \n" , double(frac),      frac.N(), frac.D());
}
```

Output:

0.75 = 3 / 4

0.75 = 3 / 4

godbolt.org/z/aPr7h8hfo

B2B: Operator Overloading – questions ?



Function & Operator Overloading – thanks for coming



B2B: Function Overloading –